AgentUCT: Budgeted Workflow Search with Reusable Execution

Supplementary Material

This supplement contains additional derivations, experimental details, complete mathematical proofs and the formal-verification map. Equations, tables and figures are numbered independently from one. References with an **M.** prefix identify items in the main manuscript; for example, Equation (M.4) is a main-paper equation.

A Additional Cost Estimation Results

A.1 The Full Evaluation Target

The conditional completion mean m_t is defined in Section M.4.

For any scalar z, the exact identity

$$\mathbb{E}[(b_t(X) - z)^2] = \mathbb{E}[(b_t(X) - m_t)^2] + (z - m_t)^2 \tag{1}$$

shows that this full conditional mean uniquely minimizes squared prediction error. The expectation covers the whole completion needed to produce one reward, including nonreusable work. It does not estimate the cost of finishing the entire optimization run.

When the actual continuation law or bills are unknown, a declared reference law gives the computable surrogate

$$\widehat{b}_t(p, a) = \sum_{x \in \mathcal{X}(pa)} \rho_{C,t}(x \mid p, a) \left[\sum_{r \in \mathcal{D}(x) \setminus K_t} \widehat{w}_{t,r} + \widehat{h}_t(x) \right]. \tag{2}$$

Uniform legal-terminal enumeration is one choice of ρ_C. A continuation model matching actual traversal and rollout is better justified for predicting (M.9). A frozen per-action calibration mean can supply $\widehat{w}_{(q,u)}$, but it discards query and prefix variation. More detailed predictors should condition on relevant execution features when supported by calibration data.

For $x = (a_1, \ldots, a_L)$, let $d_q(x, K)$ be the largest saved prefix depth, or zero if none is saved. The predicted cold bill is

$$\sum_{q \in Q} \sum_{j > d_q(x,K)} \widehat{w}_{t,(q,x_{1:j})}.$$

Current cache membership is observed. The quantities to predict are future resource demand and the cost of executing missing resources.

Checkpoint contents refine the resource-price model. If an earlier observation of (q, p) certifies that retrieval has halted, every later query/retrieval slot under p has zero model-token cost for that query, including slots whose own checkpoints have never been materialized. We therefore set

$$\widehat{w}_{t,r} = \begin{cases} 0, & \text{a prior state certifies a skipped slot,} \\ \widehat{\mu}_{t,r}, & \text{otherwise,} \end{cases} \qquad 0 \le \widehat{w}_{t,r} \le M_{a(r)}, \tag{3}$$

where M is a fixed resource-price envelope. Only observations available before selection enter this prediction. Each sample consults query-specific zero-work observations. The membership-only variant retains the online mean outside physical hits. The main loop-zero grammar has no extra retrieval rounds.

This distinguishes two effects of observed execution state. Restorability removes materialized work from the bill. A known stop predicts zero model work for unmaterialized descendants; it creates no cache hit. The no-reuse control retains the same observed stop facts while rerunning every non-skipped call. Token-free state advancement still takes measured CPU time.

Proposition 1 (Prediction and law error)**.** *Suppose* $|\widehat{w}_{t,r} - w_r| \le e_r$, $|\widehat{h}_t(x) - h_t(x)| \le e_h(x)$, *and* $|b_t(x)| \le B_{\max}$ *on the child subtree. With* $L_t = \sum_x |\rho_{C,t}(x) - \rho_{\mathrm{exec},t}(x)|$,

$$|\widehat{b}_t - m_t| \le \mathbb{E}_{\rho_{C,t}} \left[\sum_{r \in \mathcal{D}(X) \setminus K_t} e_r + e_h(X) \right] + B_{\max} L_t. \tag{4}$$

Thus a complete cost horizon removes truncation error while leaving resource and law errors to be estimated. Lower cost-prediction MSE alone does not imply a better final workflow or smaller J regret.

A.2 What Cache Accumulation Changes

The graph has two distinct effects. Physical membership determines which query-prefix states can be restored and which work is billed. The estimator reads the same membership to remove that work from each candidate's predicted bill. Changing forecast weights or the reference law changes selection scores; it does not create a physical cache hit. Executing the selected completion changes the graph and thereby changes later forecasts.

Write $c_w(K, A) = \sum_{r \in A \setminus K} w_r$. Sequential publication satisfies

$$c_w(K, A \cup B) = c_w(K, A) + c_w(K \cup A, B). \tag{5}$$

Thus the full bill from the pre-rollout snapshot already accounts for work saved as that rollout executes. The current implementation first chooses a symbolic completion, then executes and publishes its prefixes; it does not provisionally cache unexecuted previews.

There is also a precise effect on later forecasts. Holding the candidate, law, and weights fixed, publishing a completed demand A gives

$$\widehat{b}^{\mathrm{cold}}(K) - \widehat{b}^{\mathrm{cold}}(K \cup A) = \mathbb{E}_{\rho_C}\left[\sum_{r \in (\mathcal{D}(X) \cap A) \setminus K} \widehat{w}_r\right]. \tag{6}$$

The right side is the expected weight of newly saved resources shared with a later completion. It is nonnegative. Comparing successive iterations without fixing the law or predictor need not be monotone, because those quantities can also change.

Equation (2) incorporates accumulated cache state when predicting the next evaluation. It does not value an action's future cache-warming benefit over several evaluations. Such a planner would require a continuation value on the joint tree-statistics and cache state, together with an explicit remaining budget and endpoint objective. Equation (6) identifies its one-step savings component, but does not establish optimality of that planner. Richer query-prefix features could improve the current historical full-bill model. A calibrated cost lower bound would additionally support the conditional optimism statement below.

A.3 Two Short-Horizon Estimates

Let $\widehat{d}_t(p)$ be the predicted cold demand of prefix p. The historical `immediate` implementation, labeled next-edge in earlier experiments, estimates the marginal cold cost of the tree's direct child:

$$\widehat{b}^{\mathrm{next}}_t(p, a) = \max\{0, \widehat{d}_t(pa) - \widehat{d}_t(p)\}. \tag{7}$$

It is one possible cost approximation. Write a full estimate as $\widehat{b}^{\mathrm{next}}_t + \tau_t(p, a)$. If the omitted tail $\tau_t(p, a)$ is common to all candidates, subtraction changes every index by the same constant and preserves the maximizer set. Otherwise the pairwise index difference changes by $\gamma(\tau_t(p, a) - \tau_t(p, b))$. A small common tail is unnecessary; small variation across candidates is the relevant condition for preserving comparisons.

Proposition 2 (Immediate-child reduction)**.** *Under successful persistent full-panel execution, every visited child pa has its complete prefix demand cached. For nonnegative resource predictions, its penalty in (7) is zero. Consequently, this immediate-child allocation rule is equivalent to ordinary UCT allocation on the same history. Their complete trajectories agree when the initial state is identical and a shared resolver couples expansion, rollout, tie-breaking, evaluation outcomes, and stopping. In the fixed-table setting, this variant inherits the ordinary-UCT instance-dependent quality bound.*

The reduction leaves qid-prefix caching and reward backup in place. It concerns the direct-child formula (7) among initialized children. Expansion and rollout can still execute uncached suffixes, whose predicted cost can be positive. Estimating the next unpaid execution instead gives a different rule.

For example, a saved child followed by an uncached edge priced four gives zero in (7) and four in (M.13). Combining this reduction with own-cache accounting gives the immediate-child result: under the same finite fixed-table conditions, and cold-only costs or equal total recurring overhead,

$$R^{\pi}_{\mathrm{net,next}}(T) \le A^{\mathrm{UCT}}_D \log(T + 1) + B^{\mathrm{UCT}}_D + \eta W. \tag{8}$$

This upper bound follows from ordinary-UCT quality control and finite total cold work. It establishes neither a cost advantage over UCT nor endpoint-J optimality.

For a local comparison evaluated at the same state, omitted-tail variation gives a separate approximation bound. If the chosen action's omitted tail exceeds the comparator's by at most ω_t, direct subtraction gives $R^{\mathrm{local}}_{\mathrm{full}}(T) \le$

$R^{\text{local}}_{\text{next}}(T) + \eta \sum_{t \le T} \omega_t$. Hence a logarithmic proxy bound transfers when the accumulated tail variation is also logarithmic. Constant nonzero variation only supplies a linear error allowance. This same-state relation must not be substituted for the own-cache policy comparison in (8).

The historical `continuation_mean` averages $\max\{0, \widehat{d}_t(x) - \widehat{d}_t(p)\}$ over child terminals. Prefix-complete persistent caching implies $\widehat{d}_t(x) \ge \widehat{d}_t(p)$, so clipping is inactive. With a normalized reference law this equals the full cold mean minus the common parent bill. It therefore preserves the full-cold selection ordering. The implemented `acquisition_mean` uses the full bill and includes configured future overhead. Action-dependent overhead and sparse-checkpoint semantics require separate comparison; they are not covered by the common-shift identity.

Omitted recurring cost has a different consequence from finite cold work. A verified two-arm example has deterministic rewards $0.8, 0.7$, immediate costs $0.4, 0.1$, and full costs $0.4, 0.4$. With cost weight one, its full-net regret is at least $T/10 - 40 \log T - 2/5$ under the initialized next-edge UCB rule with bonus $\sqrt{2 \log(t+1)/N}$, where t counts previous pulls. The second arm appears cheaper because its recurring tail of 0.3 is omitted, although its full net value is smaller. This local recurring-cost example disproves a general logarithmic full-net guarantee for next-edge estimation. It lies outside the persistent cold-only model, and its numerical constants are not asserted for production's $c = 1.4$ tree bonus.

A.4 When Is the Index a UCB?

For a stationary bandit with known fixed action cost b_a, subtracting γb_a from the reward index is exactly UCB on the shifted returns $Y_{a,s} - \gamma b_a$. A history-dependent graph bill cannot in general be absorbed into a fixed shifted reward distribution. Nor does a UCT backup sequence become independent merely because the same algebraic index is used.

If a cost estimate has a valid radius e_a, a more principled optimistic net index uses $\underline{b}_a = \max\{0, \widehat{b}_a - e_a\}$. Given a valid reward upper bound $\mu_a \le Q_a + r_a$, it follows that

$$\mu_a - \gamma b_a \le Q_a + r_a - \gamma \underline{b}_a. \tag{9}$$

With two-sided reward error for the selected action, maximizing this index yields selected net gap at most $2r_a + 2\gamma e_a$. These are conditional optimism statements. Calibration means or empirical MAE do not supply confidence radii, and Theorem M.2 uses a separate deterministic analysis of adaptive backups. The current point-estimate implementation does not claim calibrated net optimism.

B Cache Information: Updates and Reachable Separations

The cache graph changes the bill of an evaluation through the intersection of its demand with saved resources. Whether the full incidence relation must be retained depends on the demands that future evaluations can make. A full-panel protocol and a query-selective protocol have different sufficient statistics.

Thus full-panel trials admit an exact prefix-level implementation. Query-dependent weights and nonrectangular warm caches also admit this quotient. To determine which information selective evaluation requires, we characterize a larger class of exact compressions.

The theorem identifies the coarsest partition-based residual-sum summary that works for every unit-price cache state. A single full panel has one group; the three cyclic pairs $\{q_1, q_2\}, \{q_2, q_3\}, \{q_3, q_1\}$ have three. Lossless bitsets, demand-profile tables, and incidence graphs implement the same information. The partition converse quantifies over arbitrary valid warm caches on one-step prefixes. The reachable example below establishes information loss under the overlapping-panel protocol itself. More general encodings than group sums can preserve the same information. Physical restoration still requires the saved execution objects; the quotient compresses cost accounting, not their contents. The residual-sum formula also applies to the predictor's prices at a fixed decision. A state-conditioned predictor refreshes those prices from saved execution observations. Its feature state and the residual accounting table have distinct roles; the fixed-weight publication update above concerns the latter. For live prices $\widehat{w}_t$, write $r_t(u) = \sum_q \mathbf{1}\{(q,u) \notin K_t\} \widehat{w}_t(q,u)$. After publishing $Q \times P_x$ and revising prices, the exact update is

$$\begin{aligned} r_{t+1}(u) &= 0 \quad (u \in P_x), \\ r_{t+1}(u) &= r_t(u) + \sum_{q \in Q} \mathbf{1}\{(q,u) \notin K_t\} \big(\widehat{w}_{t+1}(q,u) - \widehat{w}_t(q,u)\big) \quad (u \notin P_x). \end{aligned} \tag{10}$$

This identity, checked in `FullPanelLivePrices.lean`, separates new cache hits from new information about future execution. A saved stop state can lower an uncached descendant's predicted price. Computing that revision requires the predictor's observations. For the main fixed panel, the resulting residual totals suffice to compute completion costs under any supplied workflow law.

Proposition 3 (Query identity can be decision-relevant)**.** *There are two prefix-complete caches with the same per-prefix hit counts and the same full-panel residual bills, for which the cost-minimizing action for one fixed active query is different. No deterministic policy using only these identical summaries is optimal in both states. A randomized policy with the same action distribution in both states incurs at least* $1/2$ *unit of expected excess cold cost in one state. A single estimate of the same action's bill has absolute error at least* $1/2$ *in one state.*

Take two queries q_0, q_1, two one-step terminal prefixes u, v, unit resource weights, and caches

$$K_L = \{(q_0, u), (q_1, v)\}, \quad K_R = \{(q_1, u), (q_0, v)\}.$$

Each prefix has one hit and one unit of full-panel residual cost in either state. For a demand on q_0, the bills of (u, v) are $(0, 1)$ under K_L and $(1, 0)$ under K_R. With identical candidate quality, the preferred action reverses. Adding one unit of recurring work and imposing a remaining budget of one makes only u feasible in the first state and only v feasible in the second. This is a finite-budget information separation, not a performance theorem for the current UCT index.

Proposition 4 (Separation reachable from an empty cache)**.** *Let the allowed panels include* $A = \{q_0, q_1\}$ *and* $B = \{q_1, q_2\}$*, with two one-step execution prefixes* u, v *and unit prices. Starting empty, the complete evaluations* $(A, u), (B, v)$ *produce* $K_L = (A \times \{u\}) \cup (B \times \{v\})$*; the evaluations* $(A, v), (B, u)$ *produce* $K_R = (B \times \{u\}) \cup (A \times \{v\})$*. Both histories use the same panel order and cost four. Both have two hits and one residual unit at each prefix over the three-query universe. For active panel* A*, the completion bills of* (u, v) *are* $(0, 1)$ *in* K_L *and* $(1, 0)$ *in* K_R*. With one unit of common overhead and one unit of remaining budget, the unique affordable action reverses.*

The proof enumerates the four publications in each history and their set differences. All five claims, including reachability and equal warm-up expenditure, are checked in `ReachableDemandSeparation.lean`. This witness uses complete evaluations of overlapping panels and establishes why global prefix totals cannot serve as the cost state for that selective-demand protocol. A policy retaining each query's membership, or the equivalent demand-profile quotient, computes both states' bills exactly.

The distinction concerns information, not a prescribed data structure: a bitset or another lossless encoding of incidence is equally sufficient. A stronger prefix summary that preserves the needed conditional bills also avoids the counterexample. For query-selective evaluation, let S denote the next query subset and X the completed workflow. The prediction target is

$$m_t(p, a) = \mathbb{E}\Big[\sum_{q \in S} \sum_{u \preceq X,\, u \neq \varnothing} \mathbf{1}\{(q, u) \notin K_t\} w_{q,u} + h_t(S, X) \mid \mathcal{H}_t, p, a \Big]. \tag{11}$$

The joint law of S and X matters; separate query and path marginals need not determine their resource-inclusion probabilities. This identity follows by summing the conditional inclusion probability of each uncached resource. The full-panel estimator below sets $S = \boldsymbol{Q}$.

Executing a completion also changes later candidates' bills by publishing their shared resources. Equation (6) quantifies this effect. A full next-evaluation estimator uses the current graph; a planner that deliberately invests in reusable work additionally needs remaining budget and a value for the resulting cache state. Neither an accurate next-bill estimate nor a logarithmic quality bound alone establishes such planning optimality.

Propositions M.1–3 identify when query incidence affects cost-aware allocation. The main experiment uses the fixed-full-panel compression case. Section M.5 connects cost errors to decisions and evaluation coverage. Theorem 1 extends the quality bound to overlapping fixed panels with separate reward backups and a shared cache; this is a theoretical extension beyond the main one-tree protocol.

C Further Allocation and Budget Comparisons

The cost term prices the next completed reward evaluation. Let $U_a = Q_a + c\sqrt{\log(N+1)/N_a}$ and $\gamma = \beta / C_{\mathrm{ref}}$ at a fixed decision state. Write b_a for the expected completion bill under the declared continuation law. Equation (M.14) gives the exact exchange condition. With equal optimistic quality and $\gamma > 0$, the rule chooses a cheapest completion. With unequal quality, that condition specifies the saving required to offset the quality disadvantage. This is a resource-priced allocation rule; U_a retains its tree-exploration meaning and need not be a calibrated confidence bound on a stationary arm mean.

Proposition M.2 compares the priced choice a with the ordinary-UCT choice b at the same initialized state. If $|\widehat{b}_j - b_j| \le \epsilon_j$ for $j = a, b$, pricing bounds the sacrifice in optimistic quality and a predicted saving greater than

$\epsilon_a + \epsilon_b$ certifies a strict reduction in expected physical cost. The statement is local to one common tree and cache state. Realized stochastic bills need not follow their expectations on every call.

For a net-value interpretation, let v_j be the actual expected reward of the declared completion, and additionally require $|U_j - v_j| \le s_j$. Writing $M = (U_a - \gamma\widehat{b}_a) - (U_b - \gamma\widehat{b}_b)$ gives

$$(v_a - \gamma b_a) - (v_b - \gamma b_b) \ge M - s_a - s_b - \gamma(\epsilon_a + \epsilon_b). \tag{12}$$

A positive right side certifies a net-value improvement. The current tree bonus and measured prediction MAE do not supply these error bounds. We use Equation (M.15) to audit the implemented rule and reserve the physical and net certificates for settings with valid error bounds.

Proposition 5 (Forecast error and allocation)**.** *Let a maximize $U - \gamma\widehat{b}$, let a^* maximize $U - \gamma b$ at the same state, and suppose $|\widehat{b}_j - b_j| \le \epsilon_j$ for these two actions. For $\gamma \ge 0$,*

$$(U_{a^*} - \gamma b_{a^*}) - (U_a - \gamma b_a) \le \gamma(\epsilon_{a^*} + \epsilon_a). \tag{13}$$

The inequality follows by inserting the two estimated indices and using their ordering. Summing it bounds cumulative loss against the true-index choice at each visited state by the corresponding sum of error radii. The comparator in this statement uses the same observed state. Comparisons between whole search policies must also account for their different histories and caches.

Two consequences explain the choice of horizon and cache information. First, suppose both candidates have the same next-edge bill e, with remaining tails d_a, d_b. If $0 < U_b - U_a < \gamma(d_b - d_a)$, next-edge chooses b, while the full bill chooses a. The absolute truncation error of e is exactly d_j. Second, scale every cold bill in the two-cache witness of Proposition 3 by $H > 0$. Any allocation that sees only the identical prefix totals incurs expected excess cold cost at least $H/2$ in one of the two states. Incidence-aware allocation has zero excess in both. Thus aggregation and horizon truncation can each erase the cost difference needed by (M.14).

Proposition 6 (Coverage under separate cache histories)**.** *Consider two search policies with their own realized, nonnegative evaluation bills. Suppose each of the first m bills of the first policy is at most ℓ, and each of the first k bills of the second is at least h. If $m\ell \le B < kh$, every prefix of the first m evaluations fits budget B, whereas the second policy cannot complete its first k evaluations within B.*

Summing the respective bounds proves the claim; nonnegativity bounds each intermediate prefix cost. Bills include whatever recurring work the declared budget charges. This conditional statement gives a directly testable benefit: lower realized completion costs can fund additional reward observations. It does not order the rewards of those observations. The experiments therefore report completed evaluations, cost and final quality together, and keep estimator computation time as a separate measured resource.

Different trajectories. Let A and B be the unions of resources materialized by two policies from the same initial cache K. With fixed prices and successful persistent reuse, their cold-cost difference is exactly

$$c_w(K, A) - c_w(K, B) = c_w(K \cup B, A) - c_w(K \cup A, B). \tag{14}$$

Common materializations cancel; the policies pay differently for resources exclusive to their own trajectories. Recurring costs add their own difference. Repeated local savings therefore do not order whole-run expenditure when the policies explore different workflows. They also do not order endpoint quality. For example, take current priorities $(U_a, U_b) = (0.6, 0.7)$, exact completion costs $(1, 2)$, and $\gamma = 0.5$. UCT chooses b and pricing chooses a; both fit a budget of two. If their unknown next rewards are $(0, 1)$, pricing loses both next-reward quality and net value; for $(1, 0)$ it wins both. These two one-decision cases do not assume calibrated U errors. Lean checks the stated inequalities; they are not a constructed full-tree trajectory theorem.

The finite-budget rationale is consequently precise: use the graph to expose conditional reuse, estimate the complete reward-producing bill, and trade a bounded amount of optimistic quality for predicted savings. Evaluate the resulting policies at a common retained-search-cost ceiling, with the full optimization bill reported separately. The logarithmic theorem below establishes preservation of long-run quality exploration under persistent cold costs; it supplies no universal finite-budget advantage over UCT.

D Shared Caches, Actual Costs and Recommendation

D.1 Shared Cache across Fixed Panels

Let a fixed finite family of panels maintain separate trees and deterministic reward tables. Each tree backs up complete returns on its own panel, while all trees publish successful qid–prefix executions into one persistent cache. External publications can reduce a panel's cold forecast. They cannot remove its own previously saved resources.

Theorem 1 (Shared-cache quality regret)**.** *Under the hypotheses of Theorem M.2 for each panel, adding arbitrary external cache publications preserves its instance-dependent bound $R_j(T_j) \le A_j \log(T_j + 1) + B_j$. For a fixed finite panel family scheduled for T evaluations in total,*

$$\sum_j R_j(T_j) \le \left(\sum_j A_j\right) \log(T + 1) + \sum_j B_j. \tag{15}$$

The penalty remains nonnegative and bounded by the panel's prediction envelope $H_j = \gamma_j \sum_r M_{j,r}$. The original exploration argument eventually covers every local terminal, after which the panel's own publications already zero all its cold penalties. Adding shared entries preserves that fact. The same subtree induction proves the local bound; $T_j \le T$ gives (15). The same rate holds for a nonnegative cold-cost approximation dominated by the complete forecast under its own continuation law. Such a penalty has the same finite bound and vanishes after coverage. The first unpaid estimator has this property; its quality-regret bound follows from coverage and the tree induction, without requiring equality with UCT. The checkpoint-conditioned model (3) also satisfies this domination condition. At each history, finite-sum monotonicity bounds its cold bill by the fixed envelope M; averaging under that history's continuation law preserves the inequality. The prediction and law can both change while this envelope remains fixed. The three envelope lemmas in `DynamicCompletion` discharge the domination step. For (M.10), the calibration pseudo-observation and clipped physical observations lie in a fixed envelope, so their mean does too. The sample indices induce a nonnegative terminal law of total mass one; repeated terminals receive their accumulated mass. Lean's `online_mean_shared_root_log_bound` in `OnlineMean` connects these online prices to the shared-cache quality theorem, and its sampled-law lemmas identify (M.11) with a reference expectation. Next-unpaid is dominated by that same complete forecast. The argument uses coverage followed by exactly zero cold penalties; boundedness alone does not imply logarithmic quality regret. The bound is in completed evaluations T, and asserts neither logarithmic regret in expenditure nor a finite-budget F1 advantage. The number and definitions of panels are fixed in this result. Merging rewards from changing query subsets into one node mean requires a different analysis.

D.2 Actual Search Costs and Recommendation

We first consider one fixed reward panel with no external publications. Let $W = \sum_{r \in \mathcal{D}(\mathcal{X}) \setminus K_0} w_r$, where $\mathcal{D}(\mathcal{X})$ is the union of all terminal demands. The execution recurrence gives the pathwise conservation identity

$$C_T^{\text{cold}} = \sum_{r \in K_T \setminus K_0} w_r \le W. \tag{16}$$

This bound holds for adaptive choices. With several publishing panels, the equality applies to the family-wide cache and the sum of its physical charges. A local panel's bill alone does not equal growth of that shared cache, because other panels may have paid for the new entries. For another length-T policy π on the same fixed reward table, executing against its own cache, define the cumulative net comparison

$$R_{\text{net}}^{\pi}(T) = \sum_{t=1}^{T} [f(x_t^{\pi}) - f(x_t)] + \eta(C_T - C_T^{\pi}), \quad \eta \ge 0. \tag{17}$$

Nonnegative comparator costs imply

$$R_{\text{net}}^{\pi}(T) \le R_Q(T) + \eta \left(W + \sum_{t=1}^{T} h_t(x_t)\right). \tag{18}$$

If the two trajectories have equal total recurring overhead, that overhead cancels and the additive cost term is ηW. Thus cold-only own-cache cumulative net regret is also $O(\log T)$. Arbitrary repeated overhead can contribute a linear term.

For $T \ge 1$, the best observed training workflow $\widehat{x}_T$ satisfies $V^* - f(\widehat{x}_T) \le R_Q(T)/T$ and becomes exactly optimal once all terminals have been covered. The single-panel endpoint sorts observed terminals by that panel's training score, takes a nonempty top-k shortlist, and chooses its validation maximizer. For validation scores g, the condition $\sup_{x \in \mathcal{X}} |g(x) - f(x)| \le \epsilon$ gives

$$\max_x g(x) - g(x_{\text{end}}) \le R_Q(T)/T + 2\epsilon. \tag{19}$$

This is a deterministic table-transfer bound; the experiment does not estimate a valid uniform ϵ. Without such a condition, a training shortlist can omit the validation optimum even after full coverage. The main experiment uses $k = 5$, ranking distinct completed workflows by full-training F1 and breaking ties by first evaluation. Validation ranks

only these five candidates and never changes the search statistics. Historical multi-panel nominations are excluded from this protocol.

Finally, endpoint utility $J = q_{\mathrm{rep}}(x_{\mathrm{end}}) - \lambda_J C_{\mathrm{opt}}/C_{\mathrm{ref}}$ uses one declared reporting quality and the actual optimization bill. The formal reporting quality is held-out F1, excluding the tiny development sentinel; validation g selects the workflow. The report presents F1 at common search budgets together with J from the full physical optimization bill, including crossing and validation work. Retained-snapshot, validation, preparation and test bills are also listed separately. Its utility gap contains the cost difference even when reporting qualities are equal. C_{opt} includes the declared optimization phases, whereas (16) concerns search execution. Neither (M.18) nor (19) proves endpoint-J optimality. The selection weight β and reporting weight λ_J have separate roles.

E Generated Workflows: Cost before and after an Edit

AFlow generates a child program after choosing a parent. Its edit tree records program ancestry. Reuse depends on the child's actual invocation prefix: the query, ordered call inputs, frozen model and decoding settings. A shared parent does not establish equality of these dependencies.

Let G denote a proposal outcome containing the child program and its complete proposal/repair transcript, or a final rejection; let Z be its subsequent execution trace and $\mathcal{H}_t$ the observed history. The pre-proposal cost target is

$$m_t(a) = \mathbb{E}_{G|\mathcal{H}_t,a}\Big[c_{\mathrm{proposal}}(G) + \mathbb{E}_{Z|G,\mathcal{H}_t,a}\big[c_w(K_t, D(G,Z)) + h(G,Z)\big]\Big]. \tag{20}$$

The transcript makes the outer cost include all charged proposal and repair attempts, including different costs for identical child programs reached through different retries. Rejected proposals form failure outcomes with empty execution demand. The inner law is conditional on the generated program; it permits dependent call counts, inputs, cache misses and prices. After generation, conditioning on the observed G removes proposal uncertainty. Concrete invocation keys determine hits as execution reveals them. Known source code alone need not determine a future call's prompt, which may depend on earlier model outputs.

Equation (20) specifies the target for an open-ended edit predictor. The exploratory GSM8K pilot uses a prequential point heuristic for this target. It estimates the first proposal's tokens from the parent source and prompt length and earlier input/output receipts, then multiplies by an observed proposal-and-repair cost ratio. Its execution estimate multiplies the parent's per-query cold model-call bill by the observed ratio of child physical execution cost to parent cold cost. This ratio can reflect reused responses, new calls and retries; it does not identify an ungenerated child's invocation prefix or conditional demand law. Only edits with completed training evaluations update these statistics, including scored execution failures. An edit attempt abandoned without a completed evaluation remains charged but does not supply a fitted observation. The heuristic therefore is not an exact or calibrated estimate of Equation (20).

The retained finite-catalogue extension instead generates no proposals: its outer generation cost is zero, and the selected program fixes the logical stage sequence. Its predictor sums physical per-stage means over missing qid–prefix resources. WTB separately checks actual invocation-prefix reuse as call inputs become known. Neither predictor learns from validation or test receipts. The dynamic pilot and retained catalogue have separate budgets, configurations and evidence. Both completed pilot arms select the initial workflow and score 22/24 on the primary test, so the pilot establishes no quality improvement from generated-workflow search.

Proposition 7 (Accounting with unknown future resources)**.** *For any finite realized sequence of finite demands $D_1, \ldots, D_T$ and persistent updates $K_{t+1} = K_t \cup D_t$,*

$$\sum_{t=1}^{T} c_w(K_t, D_t) = c_w\left(K_1, \bigcup_{t=1}^{T} D_t\right). \tag{21}$$

Recurring work contributes the separate sum $\sum_t h_t$. The resource universe and future programs need not be enumerated in advance.

Induction applies Equation (5). For finite conditional scenario laws, distributing the outer and inner sums proves Equation (20); no independence assumption is needed. `DynamicCompletion.lean` checks both identities.

Proposition 8 (Conditional guarantee for a dynamic net-value rule)**.** *At each actual history let $v_t(a), m_t(a)$ be the conditional reward and full cost of one bounded edit/evaluation attempt, assigning reward zero to an exhausted attempt with no evaluation. Suppose $U_t(a) \geq v_t(a)$ and $L_t(a) \leq m_t(a)$ for every available action. Choose $a_t \in \arg\max_a \{U_t(a) - \gamma L_t(a)\}$, $\gamma \geq 0$, and let a_t^* maximize $v_t(a) - \gamma m_t(a)$ at that same history. If the chosen action's widths satisfy $U_t(a_t) - v_t(a_t) \leq s_t$ and $m_t(a_t) - L_t(a_t) \leq e_t$, then*

$$\sum_{t=1}^{T}\Big[v_t(a_t^*) - \gamma m_t(a_t^*) - v_t(a_t) + \gamma m_t(a_t)\Big] \leq \sum_{t=1}^{T} s_t + \gamma \sum_{t=1}^{T} e_t. \tag{22}$$

Optimism bounds the comparator by its index; selection bounds that index by the chosen index. Subtracting the chosen true value leaves its two widths. The one-step inequality and its sum are Lean-checked. Logarithmic loss follows when the combined width sum is $O(\log T)$. This is a conditional same-history oracle comparison. A whole-policy comparator owns a different cache history. The experimental raw-backup bonus and point cost model supply neither certified intervals nor a proved logarithmic width sum.

The finite-tree theorem uses a different route: finitely many cold resources are eventually all materialized. An unbounded edit process can demand a fresh unit-cost suffix at every step. Lean verifies that its total cold cost is exactly T, despite persistence. The open-ended setting therefore has the accounting identity and conditional width guarantee above. The empirical study uses finite RAG workflows for its main comparison. The separate dynamic AFlow pilot records the pre-proposal heuristic and realized bills at a common token budget. Neither that pilot nor the retained finite-catalogue extension supplies certified cost intervals or the width-sum condition required for the dynamic guarantee.

What WTB restores. RAG serializes the query, evidence pool, ordered answer evidence, progress and token state at each action prefix; execution forks from the deepest compatible checkpoint. The AFlow adapter starts the child Python program in a fresh workspace and restores WTB results for matching deterministic invocation prefixes. Editing a call invalidates that call and its dependent suffix; unchanged earlier calls remain reusable. This adapter does not restore Python stack frames or arbitrary program files.

Stopped RAG loops. Once a saved evidence assessment halts retrieval, remaining requested retrieval slots advance the finite grammar without issuing model calls. They have zero physical resource weight. Lean proves that adding such zero-weight demands preserves the bill. This connects the runtime guard to the fixed terminal table under its stated execution assumptions: multiple syntactic workflows may execute the same productive calls. Zero-work slots alone do not establish fixed returns across independent fresh executions.

F Experimental Protocol and Component Catalogue

The main study optimizes RAG component workflows. The finite AFlow adapter is documented as an implemented extension; its formal matrix is deferred. Every generative execution role uses the same Qwen3.5-4B checkpoint: query refinement, progress assessment, listwise reranking, answer generation, workflow review and repair. The retained finite AFlow comparison generates no proposals; the separate dynamic AFlow study generates and charges Python-workflow edits. Dedicated embedding models remain retrieval components. All inference runs on the experiment server. Up to seven GPUs run Qwen workers; the eighth hosts the shared E5, BGE and Qwen embedding service. Each trial owns its document indices and online caches; sharing model residency creates no cross-trial reuse credit. Direct small-model component selection is outside this study. The deferred planning references specify DeepSeek-V4-Pro and GPT-5.5 as offline planners, with proposed workflows executed by Qwen3.5-4B. These references currently have no completed measurements; their protocol separates planner and execution charges.

The paired RAG evaluator fixes bfloat16 inference, greedy generation in fixed query groups with model batches capped at four, encoder batches capped at eight for both index construction and query encoding, model context limits and deterministic ranking tie rules. Document indices are built under this common configuration before search. Changing these evaluator settings defines a separate experimental cohort. The completed batch cohort started all four methods and five seeds afresh; earlier single-request search trees and generated responses are not imported.

F.1 Conceptual comparison with related optimizers

Table 1 summarizes the design models discussed in Related Work. The completed five-seed RAG comparison contains AgentUCT, UCT, AgentUCT-NextC and Random. The common-space extension adds adapted AFlow, FLASH, EEIPU, LaMBO and MSBO policies and uniform-terminal search; its five-seed results are not yet complete. The separate formal GSM8K study compares AFlow, UCT and AgentUCT within the same dynamic execution system; its earlier pilot supplies execution evidence only.

F.2 Direct AFlow comparison in the common RAG space

The AFlow adapter proposes edits within the same 61,344-workflow grammar, using Qwen3.5-4B and completed training feedback. Its parent selection retains the score/uniform mixture. Proposal and repair calls enter the actual search bill, and every accepted workflow completes the full 150-question panel before it can supply feedback for

Table 1: Conceptual comparison of cost and execution-state models, not an empirical benchmark. Rows describe the cited methods' allocation mechanisms and analysis targets; Related Work supplies their references.

Method	Cost and reusable state	Allocation statistic	Analysis target
FLASH	Pipeline-prefix intermediates; LRU eviction	Cost-sensitive BO acquisition	Efficient pipeline optimization
EEIPU	Selected memoized prefixes; stage-cost GPs	EI and inverse remaining cost	Budgeted BO with memoization
LaMBO	Modular change from the preceding configuration	Slowly moving bandit and local BO	Switching-cost-regularized regret
MSBO	Inventory of partial cascades and observations	Stage models; start or advance a sample	Multistage evaluation efficiency
AFlow	Generated program and execution feedback	Parent score/uniform mixture	Workflow optimization
AgentUCT	Persistent qid–prefix demands shared by panels	Raw ancestor returns and completion-cost price	Demand-information boundary; same-state tradeoff; fixed-table quality regret

another proposal. All methods use the same three actual-cost cutoffs and Top5–validation–test selection. This comparison tests allocation within a common component space; the generated Python-workflow experiment on GSM8K tests transfer within the AFlow execution system.

AgentUCT exceeds the adapted AFlow policy in both completed paired seeds (Table 2). At 32M, the differences are 1.24 and 3.71 percentage points for seeds 11 and 23. AFlow retains 19 and 21 complete evaluations, respectively, compared with AgentUCT's 18 and 19. The observed quality differences therefore do not follow from AgentUCT completing more evaluations in these two runs. The remaining three AFlow seeds are incomplete; no five-seed competitive ranking is inferred.

Table 2: Completed paired AFlow–RAG comparisons. Entries are primary test F1 in percent under the common actual search-budget ceiling and Top5 validation protocol. Δ is AgentUCT minus AFlow in percentage points. Only completed seeds are shown; repeated budget endpoints are not independent measurements.

Seed	Budget (M)	AgentUCT	UCT	AFlow	Δ (pp)
11	16	69.51	69.44	66.25	+3.26
11	24	69.51	69.44	65.76	+3.75
11	32	69.51	69.44	68.27	+1.24
23	16	67.05	67.05	65.76	+1.29
23	24	69.52	67.32	65.76	+3.76
23	32	69.52	67.32	65.81	+3.71
37	16	69.20	69.20	63.22	+5.97
37	24	69.20	68.69	65.76	+3.44
37	32	69.20	68.69	65.95	+3.25

Complete search bills, including proposal and repair, are 32.187M and 33.429M weighted tokens for seeds 11 and 23. Their last affordable 32M snapshots cost 30.195M and 31.881M. The complete crossing steps cost 1.991M and 1.549M, whereas the amounts above the ceiling are 0.187M and 1.429M. Validation costs a further 29.319M and 20.773M; test costs 9.698M and 5.552M. All 42 and 39 rejected proposal rounds remain charged. Previously paid input-index preparation is cited separately, including the reused indices in seed 23.

F.3 Completed common-space extension measurements

Table 3 places the completed extension measurements beside the original methods for the same seed. The shared grammar, evaluator, actual-cost cutoffs and held-out selection protocol are fixed; each optimizer retains its declared initialization and allocation policy. These rows provide a direct single-seed comparison. The original four-method five-seed result remains the main quality evidence until the extension matrix is complete.

FLASH*–RAG uses two D-optimal initial evaluations, three linear-EIPS evaluations and fifteen TPE evaluations in this run. The pruned tuning domain contains 2,304 legal workflows. The asterisk denotes the declared common-space adaptation. Complete search costs 33.716M weighted tokens; validation and test cost 33.036M and 7.452M, respectively. The last complete evaluation adds 1.955M, with 1.716M above the 32M ceiling. EEIPU–RAG performs five

Table 3: Completed common-space measurements for seed 11. F1 is primary test performance in percent after Top5 validation selection; each row is one trial, without a cross-seed standard deviation. N_{32} counts completed evaluations in the last affordable 32M snapshot. Actual retained expenditure can differ below the common ceiling. Unchanged endpoints reuse complete measurements across budgets.

Method	16M F1	24M F1	32M F1	N_{32}	Retained cost (M)
AgentUCT	69.51	69.51	69.51	18	31.910
UCT	69.44	69.44	69.44	18	31.803
AgentUCT-NextC	69.51	69.51	69.51	18	31.969
Random tree	69.51	69.35	69.35	18	31.168
AFlow–RAG	66.25	65.76	68.27	19	30.195
FLASH*–RAG (TPE)	67.79	66.55	66.55	19	31.761
EEIPU–RAG	67.95	67.95	67.26	17	29.469
Uniform-terminal search	66.76	66.76	69.33	18	30.749
StaticFull	69.51	69.51	69.51	18	31.982

warmup and thirteen acquisition evaluations, with complete search, validation and test bills of 33.310M, 41.727M and 15.611M. Its runtime handoff preserves the completed search and all previously paid calls. Uniform-terminal search samples legal complete workflows without replacement and evaluates nineteen of them in this run. Its search, validation and test bills are 32.526M, 41.589M and 9.105M; the last complete evaluation adds 1.778M, taking search 0.526M above the ceiling. It uses training evaluations and the common Top5 protocol, whereas the separate fifty-workflow reference directly measures a prespecified test distribution. Shared input-index preparation is cited separately for these methods. For FLASH* and EEIPU, saved traces establish the training observations, selected paths, phase boundaries and downstream selection. They do not retain the complete acquisition rankings or random-draw transcript, so the audit does not claim to reconstruct every numerical argmax.

F.4 The RAG execution contract

The main grammar performs one retrieval round and chooses a query strategy, retriever, evidence-selection rule and retrieval width. Query choices include the current query, pseudo-document-only retrieval, Query2doc with one or three copies of the current query followed by its pseudo-document, two-query decomposition, a concise rewrite, two synonymous rewrites, and HyDE. Synonymous rewrites preserve one information need; decomposition splits it into subquestions. Multi-round retrieval is deferred to a separate lower-priority ablation.

Ordinary queries have 30 retriever options: BM25, E5, Qwen embedding, the original four-way RRF, five BGE-M3 dense/sparse mixtures with dense weights $0, 0.25, 0.5, 0.75, 1$, 15 RRF source-set/constant combinations, and six weighted fusions. The five RRF source sets are BM25+E5, BM25+BGE, BM25+Qwen, E5+BGE, and BM25+E5+BGE+Qwen; each uses $k \in \{10, 60, 100\}$. BGE denotes its dense representation in these five sets. Weighted fusion combines BM25 with E5 or Qwen. Each source's scores are min–max normalized over the candidate documents; constant scores map to zero. The BM25 weight is $\alpha \in \{0.25, 0.5, 0.75\}$ and the dense weight is $1 - \alpha$. HyDE instead selects one of three passage encoders: E5, BGE dense or Qwen. Generated passages guide retrieval and never enter the evidence pool as sources.

Evidence selection uses relevance or MMR with relevance weight $0.3, 0.6, 0.9$; retrieval width is $2, 4, 6, 10$, capped by the candidate count. The workflow then chooses identity or Qwen reranking and selects the first $2, 4, 6$ available documents, capped by the evidence count. It preserves their full source text and applies one CPU-only ordering: ranked, reversed, or ends. The ends order alternates relevance ranks between the front and back, giving $1, 3, 5, 6, 4, 2$ for six documents. All three orders preserve the selected set. For two documents, ends and ranked coincide. Ordering performs no model call; the selected ordered context is passed to one final answer-generation stage. There is no separate model-based order evaluator. Format retries follow the same charged retry contract as every other method.

The retrieval block has $A_0 = (7 \cdot 30 + 3) \cdot 4 \cdot 4 = 3408$ choices. Two rerankers, three evidence widths and three orders give $18A_0 = 61{,}344$ syntactic terminal workflows. The final answer action is uniquely determined by the selected width and adds no choice factor. Input-specific coincidences and width truncation can make different paths execute the same behavior. This count does not assert distinct effective policies. The finite-tree theorem concerns the declared terminal grammar.

The first retrieval retains the grounded evidence assessment used in the loop-value diagnostic: Qwen selects document and sentence identifiers, and the adapter recovers exact source text. Its costs and retries are recorded. Identity reranking prioritizes cited documents within the retrieved evidence; the alternative reranker uses the same Qwen model. Neither controller receives gold answers or supporting-document labels. WTB checkpoints preserve the se-

lected documents, their full text, the chosen order and the answer state. Candidates with different orders share the prefix through evidence selection and execute their own answer suffix. Compression is excluded from the main grammar.

F.5 Paper-grounded reference workflows

The earlier Agent-UCT study compared searched configurations with fixed framework presets, then examined held-out quality, chosen components and execution costs (Li et al., 2026). We retain these questions with three published pipelines. Each cited paper already combines at least two of the mechanisms represented in our grammar: Query2doc combines generated query expansion with lexical retrieval; HyDE combines hypothetical-document generation with dense retrieval; RankGPT combines candidate retrieval with listwise language-model reranking.

All references use the same candidate documents, grounded evidence assessment, one retrieval round, retrieval width six, four retained documents, ranked context and Qwen3.5-4B answer generation. Unspecified slots use the original query, BM25, relevance selection and identity reranking. The common identity reranker prioritizes documents cited by the grounded assessment, as in the search grammar. These are shared-space adaptations of the three published retrieval pipelines, evaluated as complete QA workflows.

Table 4: Three fixed references, each based on a published combination of implemented mechanisms. All are evaluated on the same 300 primary test questions without training or validation selection. Test F1 is in percent; each recipe is a single fixed-workflow measurement.

ID	Published pipeline	Fixed component choices	Test F1
P01	Query2doc–BM25 (Wang et al., 2023)	`query.query2doc:q=3` then `retriever.bm25`.	61.60
P02	HyDE–E5 (Gao et al., 2023)	`query.hyde` then `retriever.hyde_e5`.	62.89
P03	RankGPT-style BM25–Qwen (Sun et al., 2023)	`retriever.bm25` then `reranker.qwen3`.	62.53

The three adapted references reach 61.60–62.89% test F1, below the 69.57% mean for AgentUCT's selected workflows at 32M. The selected workflows improve on these specified recipes; the matched UCT comparison isolates the allocation change. Their complete test-execution bills are 3.338M, 2.971M and 5.113M weighted tokens, respectively, including one excluded sentinel per recipe. Previously paid input-index preparation is referenced separately. These measurements supply no across-run SD, and the four-contrast paired-test family awaits completion of all fifty random workflows.

Query2doc uses our Qwen JSON prompt and three query copies, compared with the original four-shot prompt, text-davinci-003 and five copies. HyDE uses one Qwen-generated passage with E5 document-side encoding, compared with the original InstructGPT and Contriever/mContriever setup. It does not average multiple generated-document embeddings; E5's training also means this adapter does not inherit an unsupervised-retrieval claim. P03 uses a single listwise window of at most six documents with Qwen3.5-4B, compared with RankGPT's BM25 top-100 candidates and GPT sliding-window protocol. Up to two format attempts are allowed, all executed attempts are charged, and two invalid orders retain the common fusion order with a recorded fallback. No retriever or reranker is retrained for this comparison.

The exact nine-action paths are frozen in `configs/rag-followup-v2/fixed-test-manifest.json`. An independent native-grammar check confirms all three are legal terminal workflows. The former twelve-recipe catalogue and additional component sensitivity sweep are outside this accelerated experiment plan.

Fifty further complete workflows are sampled uniformly without replacement from all 61,344 legal terminals, using seed 20261004 before their test execution. Every sampled workflow is evaluated; the full distribution, mean, median, spread and execution costs will be reported, without selecting a test-best workflow. This no-search random-composition reference is distinct from Random-tree allocation and from uniform-terminal search, which pays for training evaluations and uses Top5–validation selection. The three paper references and fifty random workflows require 15,900 primary question–workflow executions before reuse, plus separate sentinels.

Strong-model planning was specified for native `deepseek-v4-pro` and OpenRouter `openai/gpt-5.5`, with five plans each and high reasoning effort (DeepSeek, 2026; OpenAI, 2026). Both receive identical task instructions, grammar, component descriptions and three frozen training examples, with no test inputs or quality feedback. The first generation requests returned insufficient balance and regional unavailability, respectively. These references are deferred; provider rejection is not a zero-quality workflow measurement.

The active paired-test family contains four contrasts: the three fixed recipes and the mean across all fifty random workflows, each against the AgentUCT's mean per-question F1 vector across its five seeds at 32M. Holm adjust-

ment covers this family. The question is the paired unit; search-seed and random-workflow variation are displayed separately. Full design details appear in Section M.8.2.

F.6 Allocation and controls

Search and validation contain 150 and 300 disjoint hard questions from the official HotpotQA training split. The primary test contains 300 hard questions from the official validation split. Training and validation use sampling seed 20261003, excluding earlier search, validation, development and calibration questions; the primary test remains unchanged. Earlier mixed-difficulty searches are historical diagnostics outside this cohort. Each method and seed owns one tree over all 150 training questions. A complete workflow evaluation returns mean answer F1 over these 150 questions and backs up that single return to the expanded node and its ancestors. The tree retains its statistics, random state and qid–prefix cache between evaluations. At each retained actual-cost snapshot, completed distinct workflows are ranked by training mean F1, with first evaluation breaking ties. The top five are evaluated on 300 validation questions. Highest validation mean F1 selects the endpoint, with training rank breaking ties; 300 held-out questions provide the primary test score. Validation and test scores never enter reward backup. An unchanged workflow on an unchanged held-out panel is measured once and referenced by any repeated measurement request.

The core comparison contains random tree allocation, UCT, AgentUCT with Policy-complete costs, and AgentUCT with AgentUCT-NextC costs. UCT supplies the standard UCB-guided Monte Carlo tree search baseline. Random retains the same tree and raw-return backup but selects children uniformly, independently of their reward statistics; this is not uniform sampling over terminal workflows. Expansion and rollout remain uniform. Five paired seeds (11, 23, 37, 53 and 71) define twenty completed trials. The first eight trials were audited before continuing the remaining seeds with unchanged settings; all twenty now have saved-trajectory, accounting and endpoint-selection checks. New seeds retain the same primary questions and assess search-seed variation. If their primary scores are used to change the scientific settings, subsequent results on that panel are exploratory. Independent confirmation of such a change requires unused hard test questions fixed before execution. Each method has its own cache and tree, with the same initial state, controller, model, resource accounting and validation protocol. The demand-profile theorem also covers selective panels; this fixed full-panel experiment instantiates its compression case.

AgentUCT-NextC estimates the first positive unpaid execution per query. Policy-complete prices the complete reward using 16 legal completions sampled from smoothed historical prefix transitions. They use the same samples at a common state, isolating the omitted suffix, and share the physical-price learning rule and current cache observations. Additional allocation-stage variants, Uniform-complete and the no-cache GPU arm are deferred. A comparison of incidence and residual prefix totals can use saved states without another model run; it is not an online search requirement.

Price priors come from independent fixed-path calibration on questions excluded from search and evaluation. Each completed training evaluation updates physical cost means and prefix-transition counts. Cache hits produce no cold-price sample. The model uses qid–prefix, prefix and component means in that order, with a calibration pseudo-observation. The historical completion law approximates subsequent demand; it is not the exact current UCT policy expectation. Evidence selection and CPU ordering have zero model-token price. Width-conditioned answer prices belong to the final answer edge.

The global scale is 150 times the median calibrated one-query completion bill; tiny retains this same value. Token receipts are weighted by model parameter count in billion-parameter-token units. Batch charges use logical request tokens, including retries and excluding padding; padded slots and physical batch sizes are recorded separately. The budget is a resource proxy, with no equivalence to a fixed GPU-time or FLOP budget. Raw tokens, elapsed model time and estimator CPU time are retained. Index preparation, search, validation and held-out measurement have separate receipts. Offline checks compare recorded cost forecasts with the selected continuation's realized bill. Their difference combines continuation variability and price error; it does not isolate either source or measure an unselected continuation. No calibrated error radius is claimed.

The frozen protocol specifies five seeds and one 32M weighted-token budget, the same model context/output limits and an evaluation-count safety cap. Each tiny configuration changes only its dataset path: it loads three training queries, one validation query and one development sentinel. Formal measurement excludes that sentinel from the 300-query primary held-out score. Test F1 and EM, physical costs and distinct syntactic workflows accompany the common-budget comparison. Means and sample SDs describe variation across the five search seeds on the fixed test panel; query–seed pairs are not treated as independent repetitions.

Validation-selected primary-test F1 at a fixed search budget is the primary outcome. The search backs up training F1, uses cost-aware UCT for allocation, and selects the final workflow by validation F1 alone. As an auxiliary diagnostic, we report endpoint utility $J = F - \lambda_J C_{\mathrm{opt}}/C_{\mathrm{ref}}$ using primary held-out F1, the existing reporting weight $\lambda_J = 0.05$, and the fixed calibration scale $C_{\mathrm{ref}} = 1{,}470{,}690.0198555747$. The full physical C_{opt} includes search, its complete crossing evaluation, and validation, including failed attempts and retries. Test measurement and common index preparation have separate bills and are excluded from this declared optimization utility. A secondary snap-

shot utility substitutes the retained search bill and is labeled separately; it does not remove crossing work from the full-bill utility or physical ledger. Negative J values are possible under this fixed scale. Neither the reporting weight nor the scale is selected from test outcomes. Adding a constant would change only the displayed origin; changing the weight could change rankings. We retain the raw definition and display same-seed ΔJ against UCT in the main five-seed table. Search and endpoint selection do not optimize this reporting utility. Joint improvements in J and F1 are empirical questions, distinct from the allocation theorem. Utility comparisons use one full optimization bill per seed; a shared bill does not create independent query-level cost observations. We decompose each difference into the F1 difference and phase-specific cost contributions, further separating retained search from complete evaluations after the snapshot cutoff. The latter is the whole evaluation bill, not only the amount above the reporting budget. A validation-cost saving is part of end-to-end utility, not evidence that a search-time cost forecast was accurate. All three baseline comparisons are retained, including unfavorable differences. An unknown physical charge prevents reporting a complete utility value. This comparison concerns allocation among budgeted search methods. It does not establish that a one-off search investment is preferable to a fixed workflow with zero optimization cost.

Table 5 preserves every primary score and full-bill utility. All 214 saved adaptive selections occur at the root. This experiment does not demonstrate deeper recursive selection gains. Native-checkpoint and reuse checks cover the explicitly sampled trajectories; they do not constitute a full native-state audit of every invocation.

Table 5: All twenty 32M runs. Each entry is primary test F1 (%) / full-bill J.

Seed	AgentUCT	UCT	AgentUCT-NextC	Random
11	69.5109 / −1.251646	69.4443 / −1.242602	69.5109 / −1.284709	69.3527 / −1.189892
23	69.5228 / −1.262981	67.3162 / −1.429808	69.5228 / −1.262340	67.6926 / −1.432710
37	69.1975 / −1.164197	68.6871 / −1.157955	64.8892 / −1.200446	69.4880 / −1.165447
53	69.9657 / −1.415753	66.5768 / −1.434304	69.1490 / −1.274501	69.4550 / −1.412869
71	69.6305 / −1.456986	68.0144 / −1.470387	68.0144 / −1.480661	68.0144 / −1.474790

The five-seed AgentUCT-minus-baseline mean ΔJ is +0.03670 against UCT, −0.00978 against AgentUCT-NextC and +0.02483 against Random. Table 5 gives the five underlying utility pairs for each comparison. For the random comparison, mean contributions are +0.00765 from F1, −0.00302 from retained search, +0.01273 from complete crossing evaluations, and +0.00747 from validation. Against UCT, the F1, retained-search, complete-crossing and validation contributions are +0.01558, +0.00084, +0.01450 and +0.00578, respectively. Against AgentUCT-NextC, F1 contributes +0.01348, full search +0.00294 and validation −0.02620.

Search runs without cost-based admission checks. Every evaluation finishes and its complete physical bill is recorded. For each reporting budget B, retain the last complete snapshot with cumulative search cost at most B. Its tree statistics, cache membership and shortlist use only that trace prefix. The evaluation crossing the largest reporting budget also finishes; its work and overshoot remain in the raw trajectory, and its reward cannot enter an earlier snapshot. Validation and test measurement use each retained shortlist and have separately reported costs. Index preparation is also reported separately. The reporting budgets do not change the selection rule or interrupt a model call. The finite-tree theorem applies at the retained evaluation count $n(B)$; it does not assert $O(\log B)$ physical-budget regret.

F.7 Multi-seed summaries and reporting units

All budget summaries use the same five search seeds, with one validation-selected primary outcome per method, seed and budget. Tables report arithmetic means and sample standard deviations, $s = \sqrt{\sum_{i=1}^{5}(x_i - \bar{x})^2/4}$. The uncertainty bars in the budget and resource figures show this between-seed spread; they are not standard errors or confidence intervals. A reused endpoint keeps its original complete test measurement. Reuse across budgets does not increase the sample size.

For each budget and baseline, we subtract its primary score from AgentUCT's score for the same seed. Tables report the mean and sample SD of these five differences, together with wins, ties and losses: the numbers of positive, zero and negative saved differences. F1 is primary and EM auxiliary. These summaries describe the observed seed variation on the fixed question panel. Detailed reporting was added to the frozen runs without changing allocation or endpoint-selection parameters.

F.8 Budget-dependent held-out quality

All three cutoffs use the same five search seeds and the complete Top5–validation–primary-test procedure. Table 6 reports all method–budget means and sample SDs; the individual seed tables below preserve the complete sixty out-

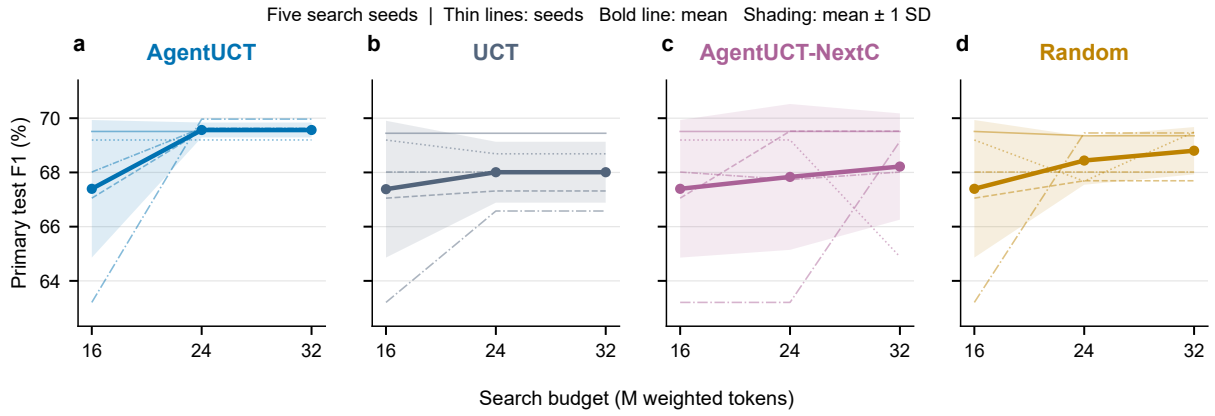


Figure 1: Individual seed trajectories behind the main budget comparison. Each method panel shows the same five seeds as thin lines and their mean as a bold line; shading is ± one sample SD. Unchanged endpoints within a trial share a complete test measurement across budgets.

comes. Early snapshots were read from frozen completed search prefixes. Only missing validation panels and newly selected primary endpoints were executed, in separate measurement caches. No later rewards, prices or execution-cache entries were imported into search. Across both early-budget campaigns, 242,293 new physical calls incurred 375.606M weighted-token units. These are additional measurement costs, not observed historical early-stopping optimization bills. We therefore do not construct 16M or 24M values of J.

Table 6: Complete matched-five-seed budget analysis. Primary test F1 and EM are reported as mean ± sample SD. Each method–budget row has five seeds; reused test measurements across budgets are not additional independent trials.

Budget (M)	Method	F1 (%)	EM (%)
16	AgentUCT	67.40 ± 2.54	52.33 ± 2.22
16	UCT	67.38 ± 2.52	52.27 ± 2.14
16	AgentUCT-NextC	67.40 ± 2.54	52.33 ± 2.22
16	Random	67.40 ± 2.54	52.33 ± 2.22
24	AgentUCT	69.57 ± 0.28	54.60 ± 0.15
24	UCT	68.01 ± 1.12	52.47 ± 1.68
24	AgentUCT-NextC	67.83 ± 2.69	53.33 ± 2.44
24	Random	68.44 ± 0.89	53.33 ± 1.49
32	AgentUCT	69.57 ± 0.28	54.60 ± 0.15
32	UCT	68.01 ± 1.12	52.47 ± 1.68
32	AgentUCT-NextC	68.22 ± 1.96	53.07 ± 1.64
32	Random	68.80 ± 0.87	53.73 ± 1.64

The eight initial root evaluations cost 12.287–16.626M weighted tokens across runs. At 16M, only seed 23 retained two completed adaptive evaluations; the other four seeds retained none, and seed 53 retained only seven initialization evaluations. Consequently, the four methods selected the same within-seed endpoint at 16M. AgentUCT, AgentUCT-NextC and Random had identical test F1; UCT's mean differed by 0.0133 points because seed 11 used a fresh execution of the same workflow. At 24M, mean F1 was 69.5655%, 68.0077%, 67.8342% and 68.4382% for AgentUCT, UCT, AgentUCT-NextC and Random, respectively. Their 32M values were 69.5655%, 68.0077%, 68.2173% and 68.8006%.

Table 7 locates the start of adaptive allocation within the actual budget. For seeds 23, 37 and 53, the cost term changes the saved AgentUCT action at its first adaptive decision; for seeds 11 and 71, the first change occurs at evaluation 11. These are comparisons with the cost-free index in the same saved state, not reconstructed UCT trajectories. Four seeds select different endpoints from UCT, with test-F1 gains of 0.51–3.39 points. Seed 11 selects the same endpoint, and its small difference comes from separate executions. The table connects initialization, allocation and endpoint changes without attributing an endpoint's F1 gain to a particular changed action.

The lack of monotonicity has a concrete selection explanation. For AgentUCT-NextC seed 53, a HyDE workflow ranked fourth in the 24M training shortlist but sixth at 32M, outside the fixed Top5. Its validation F1 of 73.0699% exceeded the other eligible winner's 72.4927%; its primary F1 was 63.2059%, versus 69.1490% for the 32M winner. Saved shortlist membership, validation order, original predictions and official scores were checked. AgentUCT-NextC

Table 7: Where adaptive allocation becomes observable. A and U denote AgentUCT and UCT. Costs are M weighted tokens. "First A feedback" is the cumulative bill after AgentUCT's first complete adaptive evaluation; "First changed choice" is the bill before its first decision whose priced argmax differs from the same-state cost-free argmax. ΔF1 is AgentUCT minus UCT in percentage points at both 24M and 32M; these budgets reuse the same endpoint measurement. Configuration IDs resolve in the workflow dictionary.

Seed	Init. A	Init. U	First A feedback	First changed choice	ΔF1	A config.	U config.
11	13.454	13.446	16.015	17.455	+0.07	C17	C17
23	12.287	12.287	13.407	12.287	+2.21	C01	C16
37	13.969	13.969	16.313	13.969	+0.51	C10	C06
53	16.626	16.626	17.362	16.626	+3.39	C15	C02
71	13.043	13.043	16.731	20.445	+1.62	C19	C05

seed 37 instead decreased from 69.1975% at 24M to 64.8892% at 32M. A larger budget can change the shortlist and validation winner without improving that winner's test performance. Neither these outcomes nor the flat AgentUCT curve was used to tune β, choose a new stopping budget or extend the search.

F.9 AgentUCT versus AgentUCT-NextC

This comparison retains the same completion sampler and price-learning rule while changing which unpaid costs enter selection. All five paired 16M endpoints coincide; three coincide at 24M and two at 32M. The paired F1 difference is 1.73 ± 2.93 points at 24M and 1.35 ± 1.79 points at 32M. These are means and sample SDs of paired *differences*, not SDs pooled across independent methods. Table 8 reports this contrast together with EM and the direction of each seed's difference.

At 32M, AgentUCT used 7.385M raw LLM tokens in its retained search versus 7.399M for AgentUCT-NextC, a 0.18% difference. Their mean logical hit rates were 6.79% and 6.27%, respectively, with the denominator defined below. AgentUCT's full search bill was 0.086M weighted tokens lower, but validation was 0.771M higher. Thus its full optimization bill was 0.684M higher, yielding the negative mean ΔJ despite higher primary F1. This separates the observed endpoint-quality result from total optimization expense and does not establish calibrated forecasts or a general cache-saving mechanism.

Table 8: AgentUCT minus AgentUCT-NextC. Differences and their sample SDs are in percentage points across five paired seeds. W/T/L counts positive, zero and negative differences; the same rows appear in Table 9.

B (M)	Contrast / metric	Δ (pp)	SD (pp)	W/T/L	n
16	NextC / F1	+0.00	0.00	0/5/0	5
16	NextC / EM	+0.00	0.00	0/5/0	5
24	NextC / F1	+1.73	2.93	2/3/0	5
24	NextC / EM	+1.27	2.48	2/3/0	5
32	NextC / F1	+1.35	1.79	3/2/0	5
32	NextC / EM	+1.53	1.71	3/2/0	5

F.10 Root allocation and selected configurations

All 214 adaptive selections in the full original trajectories occurred at the root. The retained 32M prefixes contain 194 of these; the twenty crossing evaluations account for the remainder. Tables 13 and 14 separate root-allocation counts from complete evaluation bills. Relative to UCT, AgentUCT selected the HyDE root more often (1.8±0.84 versus 1.2 ± 0.84 times after initialization) and rewriting less often (0.8 ± 0.45 versus 1.4 ± 0.55). Relative to AgentUCT-NextC, it also selected HyDE more often (1.8 versus 0.8 times). The cost attributed to a root is the entire evaluation beginning at that root, including all later components, not the cost of query construction alone. These saved allocations locate the observed policy differences; they do not identify a causal contribution of any query operator to test F1.

We report "best configurations" as validation-selected endpoints, not the best test scores among inspected workflows. The sixty selected cells contain 20 distinct configurations: five at 16M, fifteen at 24M and fifteen at 32M. At 24M and 32M all five AgentUCT endpoints are unchanged, as are all five UCT endpoints; AgentUCT-NextC changes three and Random one. Those stable cells reuse their corresponding test measurements. Across all budgets, 37 stored

Table 9: All paired budget comparisons, AgentUCT minus each baseline. Differences and sample SDs are in percentage points over five seeds. W/T/L gives the numbers of seed-level wins, ties and losses. F1 is primary and EM auxiliary.

B (M)	Contrast / metric	Δ (pp)	SD (pp)	W/T/L	n
16	UCT / F1	+0.01	0.03	1/4/0	5
16	NextC / F1	+0.00	0.00	0/5/0	5
16	Random tree / F1	+0.00	0.00	0/5/0	5
16	UCT / EM	+0.07	0.15	1/4/0	5
16	NextC / EM	+0.00	0.00	0/5/0	5
16	Random tree / EM	+0.00	0.00	0/5/0	5
24	UCT / F1	+1.56	1.33	5/0/0	5
24	NextC / F1	+1.73	2.93	2/3/0	5
24	Random tree / F1	+1.13	0.74	5/0/0	5
24	UCT / EM	+2.13	1.66	5/0/0	5
24	NextC / EM	+1.27	2.48	2/3/0	5
24	Random tree / EM	+1.27	1.44	4/0/1	5
32	UCT / F1	+1.56	1.33	5/0/0	5
32	NextC / F1	+1.35	1.79	3/2/0	5
32	Random tree / F1	+0.76	0.92	4/0/1	5
32	UCT / EM	+2.13	1.66	5/0/0	5
32	NextC / EM	+1.53	1.71	3/2/0	5
32	Random tree / EM	+0.87	1.57	3/0/2	5

Table 10: All twenty validation-selected primary outcomes at the 16M search cutoff. F1 and EM are percentages.

Budget (M)	Method	Seed	F1 (%)	EM (%)
16	AgentUCT	11	69.5109	54.6667
16	AgentUCT	23	67.0514	52.3333
16	AgentUCT	37	69.1952	54.0000
16	AgentUCT	53	63.2059	49.0000
16	AgentUCT	71	68.0144	51.6667
16	UCT	11	69.4443	54.3333
16	UCT	23	67.0514	52.3333
16	UCT	37	69.1952	54.0000
16	UCT	53	63.2059	49.0000
16	UCT	71	68.0144	51.6667
16	AgentUCT-NextC	11	69.5109	54.6667
16	AgentUCT-NextC	23	67.0514	52.3333
16	AgentUCT-NextC	37	69.1952	54.0000
16	AgentUCT-NextC	53	63.2059	49.0000
16	AgentUCT-NextC	71	68.0144	51.6667
16	Random	11	69.5109	54.6667
16	Random	23	67.0514	52.3333
16	Random	37	69.1952	54.0000
16	Random	53	63.2059	49.0000
16	Random	71	68.0144	51.6667

test measurements support the sixty reported outcomes. Identical workflows in independent runs need not share a stored test measurement.

At 32M, AgentUCT's five selected workflows use five different query actions: rewriting, decomposition, document generation, Query2doc with mixture weight three, and synonyms. All five use the identity reranker; context widths are two, four, four, six and two in seed order. This diversity shows that the observed result is not tied to a single selected pipeline, but operator frequencies are not ablation effects. The tables below list all sixty validation winners, their validation/test scores, complete component sequences and test-source identifiers. The dictionaries retain exact saved operator parameters, and repeated source codes identify shared measurements.

Table 11: All twenty validation-selected primary outcomes at the 24M search cutoff. F1 and EM are percentages.

Budget (M)	Method	Seed	F1 (%)	EM (%)
24	AgentUCT	11	69.5109	54.6667
24	AgentUCT	23	69.5228	54.3333
24	AgentUCT	37	69.1975	54.6667
24	AgentUCT	53	69.9657	54.6667
24	AgentUCT	71	69.6305	54.6667
24	UCT	11	69.4443	54.3333
24	UCT	23	67.3162	52.0000
24	UCT	37	68.6871	54.0000
24	UCT	53	66.5768	50.3333
24	UCT	71	68.0144	51.6667
24	AgentUCT-NextC	11	69.5109	54.6667
24	AgentUCT-NextC	23	69.5228	54.3333
24	AgentUCT-NextC	37	69.1975	54.6667
24	AgentUCT-NextC	53	63.2059	49.0000
24	AgentUCT-NextC	71	67.7338	54.0000
24	Random	11	69.3527	55.3333
24	Random	23	67.6926	52.3333
24	Random	37	67.6761	53.0000
24	Random	53	69.4550	54.3333
24	Random	71	68.0144	51.6667

Table 12: All twenty validation-selected primary outcomes at the 32M search cutoff. F1 and EM are percentages.

Budget (M)	Method	Seed	F1 (%)	EM (%)
32	AgentUCT	11	69.5109	54.6667
32	AgentUCT	23	69.5228	54.3333
32	AgentUCT	37	69.1975	54.6667
32	AgentUCT	53	69.9657	54.6667
32	AgentUCT	71	69.6305	54.6667
32	UCT	11	69.4443	54.3333
32	UCT	23	67.3162	52.0000
32	UCT	37	68.6871	54.0000
32	UCT	53	66.5768	50.3333
32	UCT	71	68.0144	51.6667
32	AgentUCT-NextC	11	69.5109	54.6667
32	AgentUCT-NextC	23	69.5228	54.3333
32	AgentUCT-NextC	37	64.8892	51.0000
32	AgentUCT-NextC	53	69.1490	53.6667
32	AgentUCT-NextC	71	68.0144	51.6667
32	Random	11	69.3527	55.3333
32	Random	23	67.6926	52.3333
32	Random	37	69.4880	55.0000
32	Random	53	69.4550	54.3333
32	Random	71	68.0144	51.6667

F.11 Physical tokens, crossing evaluations and cache reuse

We aggregated the physical charge identifiers already attached to each complete search evaluation. At each cutoff, only identifiers in its saved complete prefix contributed to the retained bill. Raw input/output tokens were checked against all recorded model attempts and the frozen model-size weights. Table 22 reports raw model categories, while Table 23 separates weighted retained cost from the entire first completed evaluation after the cutoff. The latter is observed in the original 32M search, not an independently rerun early-stop experiment. Additional held-out measurement bills remain separate. Table 24 reports all original phases; only full search and validation enter the declared C_{opt}.

Table 13: Adaptive root selections in retained 32M prefixes, mean ± sample SD over five seeds. Common initialization and the crossing evaluation are excluded. Subsequent expansion and rollout remained uniform.

Root query action	AgentUCT	UCT	AgentUCT-NextC	Random
decomposition	1.20 ± 0.84	1.00 ± 0.00	1.00 ± 0.00	1.40 ± 0.89
hyde	1.80 ± 0.84	1.20 ± 0.84	0.80 ± 0.45	1.40 ± 0.89
identity	1.00 ± 0.00	1.40 ± 0.55	1.20 ± 0.45	0.80 ± 0.45
query2doc	1.20 ± 0.45	1.20 ± 0.45	1.60 ± 0.55	0.40 ± 0.89
query2doc:q=1	1.20 ± 0.45	1.00 ± 0.00	1.60 ± 0.55	1.80 ± 1.30
query2doc:q=3	1.40 ± 0.55	1.20 ± 0.45	1.00 ± 0.00	1.40 ± 0.89
rewrite	0.80 ± 0.45	1.40 ± 0.55	1.20 ± 0.45	1.20 ± 1.64
synonyms	1.00 ± 0.00	1.40 ± 0.55	1.20 ± 0.45	1.40 ± 1.52

Table 14: Retained 32M cost by initiating root action, in M weighted tokens (mean ± sample SD). Each entry includes the entire evaluations in that group, including initialization. These are not component-specific query costs.

Root query action	AgentUCT	UCT	AgentUCT-NextC	Random
decomposition	3.33 ± 1.71	3.00 ± 0.80	3.20 ± 0.73	4.18 ± 1.99
hyde	4.75 ± 1.13	3.82 ± 2.56	3.06 ± 1.63	3.39 ± 1.62
identity	3.99 ± 2.55	5.87 ± 2.86	5.08 ± 1.96	3.63 ± 1.51
query2doc	4.44 ± 2.26	3.26 ± 2.90	5.02 ± 2.47	2.29 ± 1.53
query2doc:q=1	3.95 ± 1.16	3.09 ± 1.61	5.14 ± 1.49	5.70 ± 3.57
query2doc:q=3	5.34 ± 1.63	3.80 ± 1.38	3.99 ± 1.44	4.60 ± 2.71
rewrite	2.48 ± 1.21	4.00 ± 1.89	2.57 ± 1.12	3.38 ± 2.26
synonyms	2.85 ± 0.72	4.33 ± 1.30	3.13 ± 1.37	3.88 ± 2.73

Table 15: Codes used in the complete workflow dictionaries. Each numbered code denotes the exact saved action; the code sequence preserves every component and its order.

Prefix	Component
Q	Query transformation
R	Retrieval backend and fusion
S	Evidence selection
K	Retrieved-document count
C	Retrieval continuation decision
RR	Reranking
M	Context-document count
X	Context ordering
A	Answer generation

LLM input tokens account for 96.2% of retained 32M weighted search cost in each method, pooling its five seed bills. Encoder tokens and LLM output tokens are therefore reported explicitly rather than interpreting the weighted total as a count of generated answer tokens (Figure 2). The original receipts contain 300,064 model attempts without semantic `purpose`/`role` fields. Their component-edge labels identify where a bill was recorded, not necessarily the internal purpose of each call; for example, a `retrieve.k` edge can contain generation. We do not infer an exhaustive query-generation/reranking/answering split from those labels. Paid failure costs are a subset of the total, not an extra additive category, and several model attempts within an edge need not be retries.

Bipartite inventory and logical reuse. A saved key is a pair consisting of a query identifier and its complete execution prefix. We counted these objects from completed-prefix records, without instantiating an earlier cache or consulting keys introduced by later search or held-out evaluation. The 32M inventories also matched the saved materialized inventories. For this fixed full-panel design, every successful prefix covers all 150 queries, so adjacency density over the materialized prefix set is one at every measured cutoff. This is not density over all legal prefixes in the grammar. This is the rectangular special case of the information analysis, not an empirical efficiency advantage of a denser graph.

The logical hit fraction is the sum of recorded restored-prefix edges divided by the sum of all workflow-prefix

Table 16: Validation-selected RAG configurations at 16M weighted-token search budget. Rank is the auxiliary position in the saved training Top5; validation chooses the endpoint and breaks ties by that rank. F1 and EM are percentages on the fixed 300-question panels; the test sentinel is excluded. Repeated T codes reference the same stored test measurement, not independent replications.

Seed	Method	Cfg	Rank	Valid F1	Test F1	Test EM	Test source
11	AgentUCT	C17	1	74.36	69.51	54.67	T01
11	UCT	C17	1	74.63	69.44	54.33	T30
11	AgentUCT-NextC	C17	1	74.36	69.51	54.67	T10
11	Random	C17	1	74.36	69.51	54.67	T21
23	AgentUCT	C18	1	71.54	67.05	52.33	T03
23	UCT	C18	1	71.54	67.05	52.33	T32
23	AgentUCT-NextC	C18	1	71.54	67.05	52.33	T12
23	Random	C18	1	71.54	67.05	52.33	T23
37	AgentUCT	C12	1	70.46	69.20	54.00	T05
37	UCT	C12	1	70.46	69.20	54.00	T34
37	AgentUCT-NextC	C12	1	70.46	69.20	54.00	T15
37	Random	C12	1	70.46	69.20	54.00	T26
53	AgentUCT	C03	3	73.07	63.21	49.00	T07
53	UCT	C03	3	73.07	63.21	49.00	T36
53	AgentUCT-NextC	C03	3	73.07	63.21	49.00	T17
53	Random	C03	3	73.07	63.21	49.00	T28
71	AgentUCT	C05	1	71.70	68.01	51.67	T09
71	UCT	C05	1	71.70	68.01	51.67	T37
71	AgentUCT-NextC	C05	1	71.70	68.01	51.67	T18
71	Random	C05	1	71.70	68.01	51.67	T29

Table 17: Validation-selected RAG configurations at 24M weighted-token search budget. Rank is the auxiliary position in the saved training Top5; validation chooses the endpoint and breaks ties by that rank. F1 and EM are percentages on the fixed 300-question panels; the test sentinel is excluded. Repeated T codes reference the same stored test measurement, not independent replications.

Seed	Method	Cfg	Rank	Valid F1	Test F1	Test EM	Test source
11	AgentUCT	C17	1	74.36	69.51	54.67	T01
11	UCT	C17	2	74.63	69.44	54.33	T30
11	AgentUCT-NextC	C17	3	74.36	69.51	54.67	T10
11	Random	C08	2	75.05	69.35	55.33	T20
23	AgentUCT	C01	1	74.09	69.52	54.33	T02
23	UCT	C16	1	74.01	67.32	52.00	T31
23	AgentUCT-NextC	C01	1	74.09	69.52	54.33	T11
23	Random	C14	1	72.36	67.69	52.33	T22
37	AgentUCT	C10	1	71.06	69.20	54.67	T04
37	UCT	C06	1	73.65	68.69	54.00	T33
37	AgentUCT-NextC	C10	1	71.06	69.20	54.67	T14
37	Random	C13	1	72.04	67.68	53.00	T25
53	AgentUCT	C15	1	74.45	69.97	54.67	T06
53	UCT	C02	1	74.00	66.58	50.33	T35
53	AgentUCT-NextC	C03	4	73.07	63.21	49.00	T17
53	Random	C07	1	73.81	69.46	54.33	T27
71	AgentUCT	C19	3	74.60	69.63	54.67	T08
71	UCT	C05	1	71.70	68.01	51.67	T37
71	AgentUCT-NextC	C04	4	71.94	67.73	54.00	T19
71	Random	C05	1	71.70	68.01	51.67	T29

Table 18: Validation-selected RAG configurations at 32M weighted-token search budget. Rank is the auxiliary position in the saved training Top5; validation chooses the endpoint and breaks ties by that rank. F1 and EM are percentages on the fixed 300-question panels; the test sentinel is excluded. Repeated T codes reference the same stored test measurement, not independent replications.

Seed	Method	Cfg	Rank	Valid F1	Test F1	Test EM	Test source
11	AgentUCT	C17	1	74.36	69.51	54.67	T01
11	UCT	C17	2	74.63	69.44	54.33	T30
11	AgentUCT-NextC	C17	4	74.36	69.51	54.67	T10
11	Random	C08	2	75.05	69.35	55.33	T20
23	AgentUCT	C01	1	74.09	69.52	54.33	T02
23	UCT	C16	1	74.01	67.32	52.00	T31
23	AgentUCT-NextC	C01	1	74.09	69.52	54.33	T11
23	Random	C14	1	72.36	67.69	52.33	T22
37	AgentUCT	C10	1	71.06	69.20	54.67	T04
37	UCT	C06	2	73.65	68.69	54.00	T33
37	AgentUCT-NextC	C09	2	71.50	64.89	51.00	T13
37	Random	C20	1	72.61	69.49	55.00	T24
53	AgentUCT	C15	1	74.45	69.97	54.67	T06
53	UCT	C02	1	74.00	66.58	50.33	T35
53	AgentUCT-NextC	C11	1	72.49	69.15	53.67	T16
53	Random	C07	1	73.81	69.46	54.33	T27
71	AgentUCT	C19	3	74.60	69.63	54.67	T08
71	UCT	C05	1	71.70	68.01	51.67	T37
71	AgentUCT-NextC	C05	1	71.70	68.01	51.67	T18
71	Random	C05	1	71.70	68.01	51.67	T29

Table 19: Complete saved configurations, dictionary part 1. Codes are defined by component prefix and the exact operator dictionary. Configuration identifiers follow alphabetical workflow order, not quality rank.

Cfg	Complete ordered component sequence
C01	Q01 R06 S03 K01 C01 RR01 M02 X02 A02
C02	Q02 R03 S01 K01 C01 RR01 M03 X02 A03
C03	Q02 R03 S03 K01 C01 RR02 M03 X03 A03
C04	Q03 R01 S01 K01 C01 RR02 M01 X02 A01
C05	Q03 R02 S01 K01 C01 RR02 M03 X02 A03
C06	Q03 R04 S01 K01 C01 RR01 M02 X02 A02
C07	Q03 R05 S01 K01 C01 RR01 M03 X02 A03
C08	Q03 R07 S03 K01 C01 RR01 M01 X03 A01
C09	Q03 R12 S04 K01 C01 RR01 M03 X01 A03
C10	Q04 R04 S01 K01 C01 RR01 M02 X02 A02
C11	Q04 R05 S01 K01 C01 RR01 M03 X02 A03
C12	Q04 R10 S01 K01 C01 RR02 M02 X02 A02
C13	Q05 R04 S01 K01 C01 RR01 M02 X02 A02
C14	Q05 R06 S03 K01 C01 RR01 M02 X02 A02
C15	Q06 R05 S01 K01 C01 RR01 M03 X02 A03
C16	Q07 R06 S03 K01 C01 RR01 M02 X02 A02
C17	Q07 R09 S01 K01 C01 RR01 M01 X03 A01
C18	Q08 R08 S03 K01 C01 RR01 M02 X01 A02
C19	Q08 R10 S02 K01 C01 RR01 M01 X03 A01
C20	Q08 R11 S02 K01 C01 RR01 M01 X02 A01

Table 20: Exact operator definitions, part 1. These are saved workflow actions, not recommendations.

Code	Saved action
Q01	`query.decomposition`
Q02	`query.hyde`
Q03	`query.identity`
Q04	`query.query2doc`
Q05	`query.query2doc:q=1`
Q06	`query.query2doc:q=3`
Q07	`query.rewrite`
Q08	`query.synonyms`
R01	`retriever.bge_m3_hybrid`
R02	`retriever.bge_m3_hybrid:dense=0.25`
R03	`retriever.hyde_e5`
R04	`retriever.rrf`
R05	`retriever.rrf:bm25+e5+bge+qwen:k=100`
R06	`retriever.rrf:bm25+e5+bge+qwen:k=60`
R07	`retriever.rrf:bm25+e5:k=10`
R08	`retriever.rrf:bm25+qwen:k=10`
R09	`retriever.rrf:bm25+qwen:k=60`
R10	`retriever.rrf:e5+bge:k=100`
R11	`retriever.weighted:bm25+qwen:alpha=0.25`
R12	`retriever.weighted:bm25+qwen:alpha=0.5`
S01	`selection.mmr:lambda=0.3`
S02	`selection.mmr:lambda=0.6`
S03	`selection.mmr:lambda=0.9`
S04	`selection.relevance`

Table 21: Exact operator definitions, part 2. These are saved workflow actions, not recommendations.

Code	Saved action
K01	`retrieve.k=10`
C01	`control.stop`
RR01	`reranker.identity`
RR02	`reranker.qwen3`
M01	`top_m=2`
M02	`top_m=4`
M03	`top_m=6`
X01	`context.ends`
X02	`context.ranked`
X03	`context.reverse`
A01	`answer.generate:m=2`
A02	`answer.generate:m=4`
A03	`answer.generate:m=6`

visits across complete question evaluations. It includes zero-token control and ordering edges; it is not hits divided by physical LLM calls. At 32M, AgentUCT, UCT, AgentUCT-NextC and Random have mean logical hit fractions of 6.79%, 6.81%, 6.27% and 7.04%, respectively (Table 25). AgentUCT's 0.51-point increase over AgentUCT-NextC accompanies only 0.183% fewer raw LLM tokens in retained search. Its LLM tokens per complete evaluation are higher than UCT's (421.7k versus 418.1k). Differences in the selected workflow mix affect both quantities. Without a matched no-cache execution, these observations do not estimate avoided tokens or wall-clock speedup (Figure 3).

F.12 Execution variation and trace coverage

Fresh executions can differ numerically. Across three shared initialization paths and 450 question evaluations per method, UCT differed from AgentUCT on four answers, two affecting F1; AgentUCT-NextC and Random matched AgentUCT. The largest panel-mean difference was 9.52×10^{-4}, with identical recorded batch prompts and query

Table 22: Raw retained-search token counts, mean ± sample SD over five seeds. M and k denote millions and thousands. Encoder columns are input tokens; all generative roles share Qwen3.5-4B. These are observed logical request tokens, including recorded attempts and excluding padding.

B (M)	Method	LLM in (M)	LLM out (M)	E5 (k)	BGE (k)	Qwen enc. (k)
16	AgentUCT	3.167 ± 0.187	0.147 ± 0.011	26.7 ± 10.2	26.9 ± 15.3	23.9 ± 18.9
16	UCT	3.170 ± 0.192	0.147 ± 0.011	26.7 ± 10.2	26.9 ± 15.3	25.2 ± 21.0
16	AgentUCT-NextC	3.170 ± 0.191	0.147 ± 0.011	27.3 ± 10.3	27.6 ± 16.2	25.5 ± 21.5
16	Random	3.169 ± 0.191	0.147 ± 0.011	29.2 ± 11.9	29.8 ± 20.1	27.5 ± 24.9
24	AgentUCT	5.307 ± 0.156	0.210 ± 0.014	40.3 ± 16.2	48.7 ± 9.9	50.6 ± 34.5
24	UCT	5.263 ± 0.134	0.207 ± 0.012	41.2 ± 9.8	49.7 ± 10.9	41.9 ± 27.6
24	AgentUCT-NextC	5.321 ± 0.159	0.209 ± 0.014	44.9 ± 10.9	46.7 ± 13.6	52.0 ± 33.9
24	Random	5.314 ± 0.172	0.209 ± 0.014	45.8 ± 13.1	51.1 ± 17.0	48.7 ± 32.9
32	AgentUCT	7.124 ± 0.154	0.261 ± 0.017	57.2 ± 19.0	62.4 ± 13.1	66.0 ± 37.0
32	UCT	7.130 ± 0.164	0.262 ± 0.018	56.6 ± 15.8	63.8 ± 12.3	55.3 ± 34.2
32	AgentUCT-NextC	7.138 ± 0.156	0.260 ± 0.017	59.5 ± 10.3	60.4 ± 19.6	68.1 ± 44.6
32	Random	7.105 ± 0.128	0.260 ± 0.018	61.6 ± 10.9	62.7 ± 20.9	58.4 ± 34.0

Table 23: Retained search and the first complete crossing evaluation, in M weighted-token units (mean ± sample SD). N_B counts complete evaluations; Selections counts saved adaptive child-selection decisions. Crossing is its whole bill, not just the amount above the cutoff.

B (M)	Method	Retained (M)	Next crossing (M)	N_B	Selections
16	AgentUCT	13.970 ± 0.809	2.429 ± 0.903	8.2 ± 1.1	0.4 ± 0.9
16	UCT	13.982 ± 0.835	2.413 ± 0.905	8.2 ± 1.1	0.4 ± 0.9
16	AgentUCT-NextC	13.983 ± 0.833	2.468 ± 0.920	8.2 ± 1.1	0.4 ± 0.9
16	Random	13.985 ± 0.837	2.416 ± 0.931	8.2 ± 1.1	0.4 ± 0.9
24	AgentUCT	23.262 ± 0.705	1.982 ± 1.244	13.4 ± 1.5	5.4 ± 1.5
24	UCT	23.067 ± 0.587	1.875 ± 1.283	13.2 ± 1.5	5.2 ± 1.5
24	AgentUCT-NextC	23.319 ± 0.709	1.984 ± 1.240	13.4 ± 1.5	5.4 ± 1.5
24	Random	23.293 ± 0.747	1.908 ± 1.296	13.6 ± 1.5	5.6 ± 1.5
32	AgentUCT	31.142 ± 0.725	1.661 ± 0.772	17.6 ± 1.5	9.6 ± 1.5
32	UCT	31.167 ± 0.777	2.088 ± 0.653	17.8 ± 1.8	9.8 ± 1.8
32	AgentUCT-NextC	31.200 ± 0.734	1.691 ± 0.796	17.6 ± 1.5	9.6 ± 1.5
32	Random	31.054 ± 0.624	2.036 ± 0.664	17.8 ± 1.8	9.8 ± 1.8

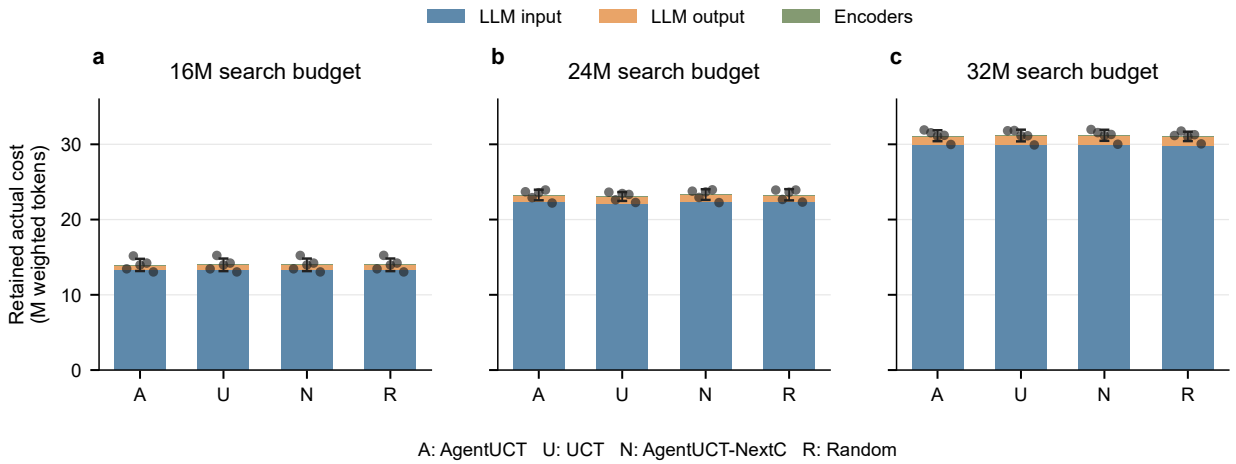


Figure 2: Physical composition of each retained search bill. Bars show five-seed means, dots show each seed's total, and whiskers show sample SD of the total. The encoder contribution is small on this common scale and is resolved in Table 22. Complete crossing and held-out bills are excluded from these bars and reported separately.

Table 24: Original 32M execution ledger by phase, mean ± sample SD across five seeds. LLM counts are raw tokens; Cost is weighted tokens, all in millions. Full search includes the complete crossing evaluation. Preparation uses encoder tokens, so its zero LLM count can accompany a positive bill. Test and preparation are excluded from C_{opt}; later snapshot measurement campaigns are separate.

Phase	Method	LLM in (M)	LLM out (M)	Cost (M)
Full search	AgentUCT	7.507 ± 0.183	0.273 ± 0.015	32.804 ± 0.808
Full search	UCT	7.613 ± 0.216	0.275 ± 0.017	33.255 ± 0.982
Full search	AgentUCT-NextC	7.528 ± 0.189	0.272 ± 0.015	32.890 ± 0.817
Full search	Random	7.576 ± 0.219	0.272 ± 0.017	33.090 ± 0.996
Validation	AgentUCT	6.041 ± 0.841	0.174 ± 0.013	26.199 ± 3.576
Validation	UCT	6.105 ± 0.856	0.156 ± 0.015	26.369 ± 3.650
Validation	AgentUCT-NextC	5.867 ± 0.812	0.169 ± 0.023	25.429 ± 3.493
Validation	Random	6.095 ± 0.878	0.171 ± 0.010	26.419 ± 3.710
Test	AgentUCT	0.926 ± 0.091	0.035 ± 0.007	4.061 ± 0.422
Test	UCT	1.123 ± 0.448	0.028 ± 0.007	4.847 ± 1.884
Test	AgentUCT-NextC	1.147 ± 0.435	0.029 ± 0.007	4.959 ± 1.826
Test	Random	1.105 ± 0.458	0.029 ± 0.008	4.779 ± 1.921
Preparation	AgentUCT	0.000 ± 0.000	0.000 ± 0.000	1.792 ± 0.000
Preparation	UCT	0.000 ± 0.000	0.000 ± 0.000	1.792 ± 0.000
Preparation	AgentUCT-NextC	0.000 ± 0.000	0.000 ± 0.000	1.792 ± 0.000
Preparation	Random	0.000 ± 0.000	0.000 ± 0.000	1.792 ± 0.000

Table 25: Cache inventory and observed work, mean ± sample SD over five seeds. Keys are distinct query–prefix pairs, Prefixes counts distinct full prefixes, and Visits counts logical prefix accesses. Hit is a per-run ratio averaged across seeds. LLM/eval is actual input plus output tokens per complete 150-question evaluation. Densities over the observed prefix sets are one.

B (M)	Method	Keys (k)	Prefixes	Visits (k)	Hit (%)	LLM/eval (k)
16	AgentUCT	11.01 ± 1.36	73.4 ± 9.0	11.07 ± 1.48	0.44 ± 0.99	408.3 ± 45.9
16	UCT	11.01 ± 1.36	73.4 ± 9.0	11.07 ± 1.48	0.44 ± 0.99	408.6 ± 45.6
16	AgentUCT-NextC	11.01 ± 1.36	73.4 ± 9.0	11.07 ± 1.48	0.44 ± 0.99	408.6 ± 45.6
16	Random	11.01 ± 1.36	73.4 ± 9.0	11.07 ± 1.48	0.44 ± 0.99	408.6 ± 45.6
24	AgentUCT	17.19 ± 1.86	114.6 ± 12.4	18.09 ± 2.05	4.92 ± 1.36	415.6 ± 43.5
24	UCT	16.95 ± 1.80	113.0 ± 12.0	17.82 ± 2.00	4.82 ± 1.09	418.5 ± 45.3
24	AgentUCT-NextC	17.28 ± 1.82	115.2 ± 12.1	18.09 ± 2.05	4.40 ± 0.83	416.6 ± 43.8
24	Random	17.34 ± 1.79	115.6 ± 11.9	18.36 ± 2.05	5.45 ± 2.07	410.0 ± 44.6
32	AgentUCT	22.14 ± 1.82	147.6 ± 12.1	23.76 ± 2.05	6.79 ± 0.82	421.7 ± 30.9
32	UCT	22.38 ± 2.14	149.2 ± 14.2	24.03 ± 2.41	6.81 ± 1.23	418.1 ± 34.8
32	AgentUCT-NextC	22.26 ± 1.80	148.4 ± 12.0	23.76 ± 2.05	6.27 ± 0.66	422.5 ± 30.9
32	Random	22.32 ± 2.05	148.8 ± 13.7	24.03 ± 2.41	7.04 ± 1.56	416.7 ± 36.0

order at the differing answer. Within one run, a successful terminal checkpoint supplies its saved prediction; the sampled main trajectories contained no repeated complete terminal. The quality-regret theorem assumes a fixed evaluator table, whereas these empirical comparisons retain observed variation between fresh executions. After the initial stage, an observer-copy repair and a retrieval-service memory reservation optimization were deployed with separate provenance. Fixed-request checks preserved rankings and token charges, without establishing global bitwise equivalence across fresh runs.

The implementation must pass evidence-order, query-history, stopping, checkpoint-restoration, physical-price update and recovery regressions. Model-attempt records retain checkpoint, prompt, output, input/output tokens and available timing/failure metadata. Semantic role labels were absent from the original receipts, as noted above. Candidate traces retain raw backup statistics, probabilities, current cost forecasts and selected actions. Search and tiny queues perform no trace-reconstruction audit. Offline audits check saved candidate arithmetic, backup, predictions, checkpoints and bills directly; they do not rerun search, random draws, historical forecasts or generation. Prediction calibration requires separate evidence. The runtime journal persists physical receipts and completed items; the post-hoc accounting path does not scan all earlier response bodies before a component call. Four query workers process fixed consecutive QID groups inside one candidate. Before each model dispatch, every unfinished group member

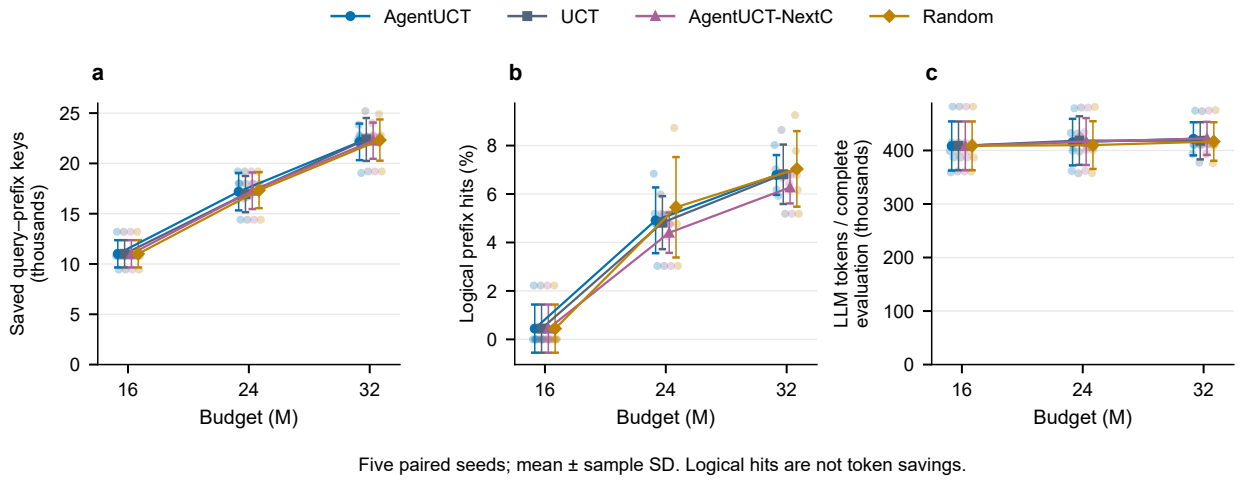


Figure 3: Saved objects, observed logical reuse and actual LLM work have different interpretations. Markers/lines show means, whiskers show sample SD, and faint points show the five seeds. Horizontal offsets separate methods at the same budget; they are not different budgets. A lower SD whisker can extend below zero because SD describes spread rather than a probability interval. Cache hits are not a measured counterfactual saving.

either requests generation or finishes; requests with equal output limits are batched in stable QID order. Cached or failed items leave the group explicitly, and all submitted work drains before an error propagates. Provider observers and receipts are per-query; retrieval cache reset and ranking are atomic. All 150 training items settle before scoring, price updates and the next search decision. Call elapsed times may include component queueing; queue time and component execution time are separately recorded, and their sums are not wall-clock time. Top-m selection precedes CPU ordering; ordering preserves selected text, and the final answer stage generates the workflow's answer.

Runtime diagnostics count each physical generation batch once, using its session and batch identifiers. Padding efficiency is the ratio of actual tokens to padded tensor positions, reported separately for inputs and outputs. Batch timings from a recent receipt window diagnose execution overhead; end-to-end speed comparisons require completed, matched workloads.

New and resumed follow-up RAG runs reuse the native checkpoint already read when a new prefix completes, removing a second checkpoint read and graph compilation. Cache hits and process recovery still read durable checkpoints. The saved runtime record identifies the implementation used for each launch; search settings, fixed batches, checkpoints and actual bills are preserved.

Earlier multi-round runs remain archived diagnostics. Independent calibration of that source pool used 34 fixed paths on three separate questions, producing 102 evaluations, 933 component receipts and 383 Qwen generation attempts. The shared calibration bill was 1,171,878.42 weighted-token units: 1,165,642.43 for execution and 6,235.99 for index preparation. Its recorded 425.44 seconds include preparation and output persistence and exclude model loading. The main grammar retains the corresponding query/retrieval/reranking forecasts and transfers the former width-conditioned answer prices to explicit final-answer edges. Removed compression actions receive no mass in the new completion law. These transferred price priors and the common numeric scale remain fixed across methods and tiny; they are not calibrated error bounds. Main-runtime acceptance additionally checks the complete one-tree, raw-backup, cache and Top-5 protocol, as well as complete token/time records on the new trajectories. The frozen parser accepts a singular `citation` key as an alias for `citations` and closes one missing final object brace only when the result is otherwise valid JSON. It still validates citation identifiers and rejects unfinished strings or arrays. Raw output and each normalization are retained. The revised calibration has no query fallback.

F.13 Exploratory dynamic AFlow pilot

The completed GSM8K pilot uses LLM-generated Python edits. Two arms pair seed 11: a bounded adaptation of AFlow's original allocator and a dynamic-edit adaptation of AgentUCT. Both use Qwen3.5-4B, the upstream proposal template with its XML output contract, the actual operator interfaces and the official scorer. Each workflow is evaluated on eight training questions before a tree update. The two best distinct training workflows enter an eight-question validation panel; its winner is measured on 24 primary questions, excluding one separate sentinel. The panels are disjoint, but were previously sampled for development.

The single reporting budget is 100k raw input-plus-output tokens, with caps of 32 complete workflows and 32 proposal rounds, each allowing up to three proposal or repair attempts. Complete evaluations can cross the budget; the

reported snapshot is the last affordable complete state. The full bill retains that crossing, proposals, repairs, validation and paid failures. Test costs are separate. The diagnostic utility is $J_{\text{pilot}} = F - 0.05C_{\text{opt,full}}/8192$. The inherited diagnostic scale and $\beta = 2$ are not independently calibrated; this utility is not pooled with RAG's parameter-weighted-token utility.

AFlow samples eligible top-four parents with probabilities $0.3/K + 0.7\,\text{softmax}(20s)_i$. AgentUCT uses progressive widening, raw descendant-return backup and a prequential estimate of complete edit cost from earlier completed training proposals, repairs and executions. The widening limit is $\min(8, \max(1, \lceil\sqrt{N}\rceil))$, with maximum edit depth 12, exploration coefficient 1.4 and no rollout. Proposals use temperature 0.7 and top-$p = 0.9$; execution is greedy. The context cap is 8192 tokens, with output caps of 2048 for proposals and 1024 for execution. Each question–workflow execution permits ten model-request slots, counting retries and cache replays. Each formatted-model invocation permits up to three format attempts within that limit; an operator can make several such invocations. A 600-second deadline is checked between requests. An in-flight synchronous model call settles before timeout handling, so this is not a hard wall-time cap. Completed pilot receipts cover 276 physical calls and 251 native WTB result checkpoints. A generated Programmer workflow executed Python on all eight training questions. Neither arm recorded a model-response cache hit, so this pilot does not measure reuse gains. Both selected the initial workflow. The parent eligibility, single training evaluation, bounded repair, feedback and program deduplication differ from upstream AFlow. The comparison therefore tests these bounded policy adaptations, not a complete upstream reproduction or the cost term alone. Generated code has unknown future demand; the estimator supplies no calibrated error bound. The finite-terminal quality theorem does not automatically apply to evaluable and expandable programs.

Each arm owns its execution cache and ledger. Python reruns while WTB restores saved model responses at matching complete invocation prefixes. Successful edits, adaptive selections, actual Programmer execution, failures, the full bill and validation-to-test selection have been audited in the completed pilot. No prefix cache hits occurred, so the observed run supplies no reuse-benefit evidence.

F.14 Static complete-cost control and selection depth

The static control tests whether dynamic adaptation improves on a fixed complete-cost prior. Before its search begins, independent calibration fixes the same resource-price priors and caps as AgentUCT, giving prices $\bar{w}_r$. Any estimator overhead $\bar{h}(p, a)$ is fixed at its pre-search value. The continuation law $\bar{\rho}(\cdot \mid p, a)$ is the original estimator's zero-history law: each successive legal child is equally likely, which does not generally make complete terminals equally likely. The original 16 completion samples and smoothing are retained. A private seed–state–action random tape fixes the sampled completions without advancing search randomness. The frozen target is

$$\bar{b}(p, a) = \mathbb{E}_{X \sim \bar{\rho}(\cdot|p,a)}\left[\sum_{r \in \mathcal{D}(X)} \bar{w}_r\right] + \bar{h}(p, a),$$

and is estimated by the fixed mean of 16 sampled completions, using an empty workflow-execution cache as the reference. Structural zero-cost steps remain free. Shared preparation retains its original separate account. Prices, continuation probabilities and the sampled completions for each state–action remain fixed; the estimator reads neither live cache membership nor later execution observations. Previously unseen states are priced lazily from these frozen inputs, without tree enumeration, historical reconstruction or consumption of the search random stream.

Execution still restores and extends the actual WTB cache. Raw F1 backup, initialization, exploration, rollout, β, C_{ref}, models, panels, physical batching and Top5–validation–test selection match AgentUCT. Five additional static-control searches use seeds 11, 23, 37, 53 and 71 and report the same 16M, 24M and 32M actual-cost cutoffs. Only the price used in selection is held static; actual failure, retry and crossing charges remain billed. This intervention evaluates dynamic cache conditioning, price learning and continuation adaptation jointly. It does not distinguish their individual effects. In the completed seed 11, StaticFull and AgentUCT select the same workflow at all three budgets and attain the same 69.51% test F1 (Table 3). At 32M, both retain eighteen evaluations; StaticFull's retained search bill is 31.982M weighted tokens. This seed supplies no additional endpoint-quality gain from dynamic pricing. The remaining four paired seeds are required to assess the average effect.

StaticFull completes nineteen training evaluations in this run, costing 33.693M including the crossing evaluation. The complete crossing step costs 1.711M; the amount above 32M is 1.693M. Validation and test add 39.807M and 3.641M, respectively. Previously paid input-index preparation is cited separately. Equal selected endpoints across budgets reuse a single complete test measurement and do not provide three independent comparisons.

The implementation retains the original zero-history sampling tape and prices the entire root-to-terminal evaluation, including its selected tree prefix, against the empty cache. Static observation hooks skip receipt-based price and transition updates; the execution ledger still records every actual call. A separate run namespace preserves the original dynamic-control results.

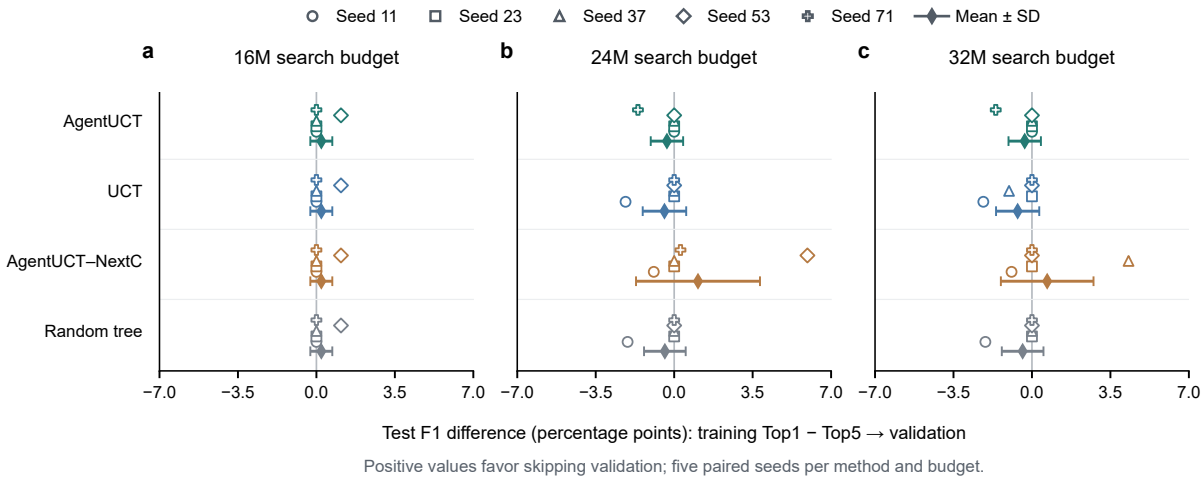


Figure 4: Complete effect of skipping validation. Positive paired test-F1 changes favor training Top1. Small symbols retain each seed; the lower filled diamond and horizontal bar show mean ± sample SD across five seeds. Both rules use the same search prefix. Repeated budgets can reuse endpoints and are not independent replicates.

The saved RAG evidence in Table 7 connects initialization cost, first complete adaptive feedback, cost-induced selection changes and paired endpoint effects. In the partial formal GSM8K seed-11 trace, completion pricing also changed a decision below the root. Through eighteen completed AgentUCT training panels, twenty recorded traversal decisions included eight below the root, two at selection-state depth two. The cost term changed sixteen choices overall, including five below the root. The corresponding saved UCT prefix contained fourteen completed panels and ten traversal decisions, one at depth one. These are incomplete search prefixes, with no formal endpoint test result.

Table 26 shows the cost-induced depth-one decision. At parent node 5, the quality-plus-exploration index favoured node 7. Its estimated complete edit-and-evaluation cost was 1.382M raw tokens, compared with 0.249M for node 6, so AgentUCT selected node 6 as the next edit's input. The resulting new program, node 9, scored zero on its complete 150-question training panel. The trace therefore establishes that completion pricing affected recursive allocation; it supplies no quality-gain evidence for this particular choice. Candidate statistics are taken from the contemporaneous saved traversal. Selection-state depth follows saved parent links and is distinct from the depth of the generated program.

Table 26: A saved depth-one AgentUCT decision in the partial formal GSM8K seed-11 run, edit 9. Both candidates are children of node 5. The cost-free index selects node 7; subtracting $2\widehat{b}/C_{\mathrm{ref}}$ selects node 6, with $C_{\mathrm{ref}} = 150{,}954.5$ raw tokens. The edited child subsequently scored zero on the training panel; no endpoint-quality claim follows from this case.

Node	N	Q	Bonus	Q + bonus	Est. cost (M)	Net index
6 (selected)	2	0.3067	1.2559	1.5626	0.2494	−1.7424
7	1	0.2533	1.7761	2.0294	1.3822	−16.2831

F.15 Complete training-Top1 measurements

The no-validation ablation covers all sixty method–seed–budget cells. For each completed affordable search prefix, the highest training mean F1 selects the endpoint, with the earliest evaluation breaking ties. Its paired selection effect is

$$\Delta_{s,B}^{\text{selection}} = F_{s,B}^{\text{train Top1}} - F_{s,B}^{\text{Top5}\to\text{validation}}.$$

Forty-nine cells reuse complete tests of the exact workflow from the same original run. Eight additional method–seed workflow measurements, involving four distinct configurations, supply the remaining eleven cells. They require 2,400 primary question–workflow executions and eight sentinels. Each prediction was rescored with the official evaluator and bound to its saved execution and terminal receipt. The new measurements contain 21,680 settled component/preparation records and 5,791 generation attempts, costing 29.722M weighted-token units in the separate ablation-test account. They add no search observations and do not change historical snapshots.

At 16M, skipping validation raises all four method means by 0.22 percentage points, entirely from seed 53. At 32M it lowers AgentUCT by 0.32 points, UCT by 0.64 points and Random tree by 0.42 points, while raising NextC by 0.68

Table 27: Complete selection ablation. Test F1 is in percent; paired Δ is in percentage points, positive for training Top1. Scores and differences are mean ± sample SD across five matched seeds. Win/tie/loss compares endpoint-selection rules within a seed.

Budget	Method	Train Top1	Top5 → validation	Paired Δ	Win/tie/loss
16M	AgentUCT	67.61 ± 2.09	67.40 ± 2.54	+0.22 ± 0.49	1/4/0
16M	UCT	67.60 ± 2.08	67.38 ± 2.52	+0.22 ± 0.49	1/4/0
16M	AgentUCT–NextC	67.61 ± 2.09	67.40 ± 2.54	+0.22 ± 0.49	1/4/0
16M	Random tree	67.61 ± 2.09	67.40 ± 2.54	+0.22 ± 0.49	1/4/0
24M	AgentUCT	69.24 ± 0.74	69.57 ± 0.28	−0.32 ± 0.72	0/4/1
24M	UCT	67.57 ± 0.80	68.01 ± 1.12	−0.43 ± 0.97	0/4/1
24M	AgentUCT–NextC	68.90 ± 0.59	67.83 ± 2.69	+1.06 ± 2.77	2/2/1
24M	Random tree	68.02 ± 0.84	68.44 ± 0.89	−0.42 ± 0.93	0/4/1
32M	AgentUCT	69.24 ± 0.74	69.57 ± 0.28	−0.32 ± 0.72	0/4/1
32M	UCT	67.37 ± 0.53	68.01 ± 1.12	−0.64 ± 0.96	0/3/2
32M	AgentUCT–NextC	68.90 ± 0.59	68.22 ± 1.96	+0.68 ± 2.07	1/3/1
32M	Random tree	68.38 ± 1.03	68.80 ± 0.87	−0.42 ± 0.93	0/4/1

points. Across the sixty correlated cells, seven improve and nine decline; the other 44 select the same workflow under both rules. AgentUCT retains the highest mean at 24M and 32M under either rule. Other method rankings change: NextC overtakes Random tree at 32M and both UCT and Random tree at 24M. These changes are descriptive; no test outcome changes the primary Top5–validation protocol. Omitting validation avoids its physical bill, while the extra test measurements remain a separate research expense.

G Algorithm Adapters in the Shared RAG Grammar

The direct comparison holds the executable workflow family fixed and changes how the next evaluation is chosen. This section specifies the implemented adapters behind Table M.4. It documents an experimental protocol; the pending rows have no completed five-seed quality estimate. The original algorithms motivate the selection rules, while the categorical representation, RAG observations and shared execution service define their use in this study.

G.1 Common representation, feedback and accounting

Write a legal workflow as $x = (a_1, \ldots, a_9)$. Its actions select the query, retriever, relevance or diversity rule, retrieval width, stopping rule, reranker, retained evidence count, context order and answer generator. The no-loop grammar has 61,344 terminal tuples. A candidate is legal only if every action is admitted after its preceding prefix; this enforces the query–retriever coupling for HyDE and the matching evidence-count action at generation. Candidate sets contain only legal terminals. Complete workflows already evaluated within a run are excluded from further objective evaluations.

The scalar objective is mean answer F1 over the fixed 150-question training panel. Complete-workflow optimizers receive that value only after every question has finished. EEIPU and LaMBO use three modules ending after actions 1, 6 and 9: query construction, evidence construction, and context and answer generation. These module boundaries organize the surrogate; their objective observation remains a complete-panel F1. MSBO instead executes two stages ending after actions 6 and 9, with an intermediate observation explicitly defined below. No optimizer receives validation or primary-test feedback during search.

All runs use the same persistent WTB prefix service and actual weighted-token accounting. A prefix is declared available to a panel-level cost model only when it is materialized for all 150 training questions. Reusing a prefix restores its execution state; it does not create a new objective measurement. Recorded physical receipts provide both the new incremental bill and, where needed, an earlier observed cold-stage cost. Reading a previous receipt for surrogate fitting does not charge it again. Input-document indexing is common preparation and is reported separately; sharing those prepared indices transfers no predictions, search history or executed workflow prefixes between runs.

For a reporting budget b, let t_b be the last complete terminal-workflow boundary with cumulative search charge $C_{t_b} \le b$. Training Top5, the cache view and optimizer history all come from that boundary. Validation then chooses one of those workflows for primary-test evaluation. Earlier snapshots cannot use a later observation, cache entry or surrogate update. The raw account retains all intermediate stages and the full crossing evaluation. Initialization, proposal generation, repairs and paid failures are included when the method incurs them. Validation, test and shared

preparation remain separate cost categories. Resource-dependent schedules use the largest reporting budget, B = 32M, throughout a run; the 16M and 24M results are snapshots of that same trajectory.

Table 28: Fixed adapter settings. Counts refer to paid complete training workflows unless stated otherwise. A model-based candidate set is evaluated by the acquisition function; only its selected workflow is executed on the training panel.

Adapter	Initialization and allocation	Fixed numerical settings
AFlow–RAG	One BM25 workflow; top-four parent mixture; Qwen edits	Uniform weight 0.3; softmax weight 0.7 and scale 20; temperature 0.7; top-p 0.9; three attempts per round; 2,048 output tokens
FLASH*–RAG (TPE)	D-optimal design, linear EIPS, then pruned categorical TPE	Phase thresholds $0.15B$ and $0.30B$; at most 1,024 design signatures; ten pruning signatures; ridge penalty 1; EI margin 0.01; TPE $\gamma = 0.25$
EEIPU–RAG	Five warmup workflows; Top5-prefix and empty-prefix proposals	Candidate target 1,024; 128 cost Monte Carlo draws; acquisition floor one weighted-token unit; cost exponent $\max(0, 1 - C/B)$
LaMBO–RAG	Five warmup workflows; modular partition tree and lazy updates	Tree depths $(1, 1)$; local UCB $\mu + 2\sigma$; bandit learning rate 0.05; fixed categorical partitions
MSBO–RAG	Five complete warmup workflows; evidence-start or saved-prefix advance	256 new-path candidates; 256 cascade draws; EI fallback threshold 0.005; $\beta = 1 + 100\max(0, 1 - C/B)$; full-path window $B/105$
Uniform-terminal search	Uniform terminal ranks without replacement	Same five search seeds and maximum 256 complete workflows

G.2 AFlow: parent selection, feedback and legal edits

The adapter retains AFlow's score-weighted parent mixture and experience-guided proposal mechanism (Zhang et al., 2024). Its initial workflow uses the original query, BM25, relevance selection, retrieval width six, no additional retrieval round, identity reranking, four retained documents and ranked context. This initial workflow receives the same complete training evaluation and bill as a proposed workflow.

Completed workflows are sorted by their own training F1, with earlier evaluation order breaking ties. If K eligible workflows are available, the parent pool contains the best $\min(4, K)$. Denoting its size by k, the probability of choosing parent i is

$$p_i = \frac{0.3}{k} + 0.7\frac{\exp(20F_i)}{\sum_{j=1}^{k}\exp(20F_j)}.$$

These scores are the parents' own evaluations. Descendant returns are not backed up into the AFlow parent score. The selected parent's recorded modifications, resulting scores and unsuccessful proposals form its experience history.

The Qwen editor receives that history, the parent tuple and the legal grammar. Training feedback contains up to three examples: the first imperfect predictions in frozen panel order, or the first examples if all predictions receive F1 one. Each example includes the question, prediction, training gold answer and per-question F1. Questions and predictions are limited to 800 and 400 characters, respectively. Thus the editor is allowed supervised training feedback, while held-out outcomes remain inaccessible. The prompt requests one component edit, allowing coupled changes needed for HyDE or matching evidence-count actions. The executor's task prompts and model weights cannot be edited.

A response must contain a nonempty modification description and a complete action tuple. Acceptance checks enforce grammar legality, reject a terminal tuple already evaluated in the run, and reject an identical modification description previously attempted from the same parent. A failed check is returned to the editor for repair, with at most three generated responses in total per round. This is a bounded repair policy, rather than a new training evaluation for an invalid graph. All paid responses remain in the search bill. A round exhausted by invalid responses creates no F1 value. The implementation caps both proposal rounds and complete training evaluations at 256, counting the initial workflow among the latter.

Proposals use temperature 0.7, top-p = 0.9 and at most 2,048 output tokens. For reproducibility, proposal attempt a in round r has random seed $(s + 1009r + 9176a) \bmod 2^{31}$. Model execution for task answers retains the common greedy policy. Replacing Python graph edits by finite component tuples, restricting feedback to these examples and using one fixed-panel evaluation per valid tuple are declared adaptations of AFlow.

G.3 FLASH*: design, structural pruning and categorical TPE

FLASH separates algorithm choice from conditional parameter tuning (Zhang et al., 2016). We form a structural signature by retaining component families and fusion sources while removing numeric settings such as retrieval width, retained evidence count, mixture weights and diversity parameters. A second representation retains every categorical action and its setting. Both representations are one-hot encoded.

The first workflow is uniform over legal terminals. While cumulative actual cost is below $0.15B = 4.8$M, or fewer than two observations exist, the design step greedily increases the design Gram matrix's pseudodeterminant. It considers at most 1,024 randomly sampled structural signatures when the available set is larger. Numeric settings are drawn uniformly among unseen legal workflows with the selected signature. The finite online design replaces the source benchmark's initialization schedule.

From $0.15B$ to $0.30B = 9.6$M, additive ridge models predict F1 and log observed execution cost from the structural encoding. Both use penalty one and an unpenalized intercept. Let $\mu_F(x), \sigma_F(x)$ be the quality prediction and $\widehat{\ell}_C(x)$ the fitted log-cost prediction. The source-code EIPS convention implemented here is

$$A_{\mathrm{FLASH}}(x) = \frac{\mathrm{EI}(\mu_F(x), \sigma_F(x); F^*, \xi)}{\max(\widehat{\ell}_C(x), 10^{-6})}, \qquad \xi = 0.01.$$

In particular, the denominator is the predicted log-cost used by the authors' public implementation; it is not exponentiated into expected seconds. Costs entering the logarithm have a one-unit floor. These floors protect the predictor and leave physical zero-cost receipts unchanged. The maximizing structural signature is selected, then an unseen legal setting within that signature is sampled uniformly.

At $0.30B$, EIPS with $\xi = 0$ ranks all structural signatures. The best ten define the retained choices at each component position. Their Cartesian closure, intersected with the legal grammar, forms the pruned graph; it can contain more than ten complete structural combinations. The remaining budget uses the FLASH* TPE variant. Hyperopt's categorical lower- and upper-loss densities use $\gamma = 0.25$, prior weight one and its linear forgetting weights. The candidate score is the sum of log density ratios over categorical variables, maximized over unseen legal points in the pruned graph. Query and retriever form one joint variable so that HyDE constraints hold. Other variables encode selection, retrieval width, reranking, retained evidence and context order. Ties are sampled uniformly.

The 15%/15%/70% phase proportions are actual-cost thresholds, not numbers of workflow evaluations. Every selected evaluation finishes before a phase transition. The common persistent cache replaces FLASH's native cache management, and exhaustion of the pruned finite graph ends search. These choices define FLASH*–RAG (TPE), distinct from a replication of a FLASH–SMAC experiment.

G.4 EEIPU: measured stage costs and memoized prefixes

EEIPU combines objective improvement with uncertain remaining execution cost (Essofi et al., 2024). After five uniformly drawn, distinct complete warmup workflows, it fits one GP to F1 and three GPs to log stage costs. Stage costs come from saved physical component calls for the same question and action prefixes, including recorded failed attempts. If a prefix is reused, its previously observed cold cost can train the surrogate while the current incremental bill remains zero. An unobserved cold cost is not imputed as a zero-cost observation.

Candidate generation uses the empty prefix and the distinct available nonterminal module prefixes of the five best training workflows. With P such prefixes, each supplies up to $\lceil 1024/P \rceil$ uniformly sampled unseen legal completions, and duplicates are removed. The resulting pool is approximately 1,024 candidates; overlap and finite completion sets can reduce it. The empty prefix permits changes to every component.

For candidate x, let $\widetilde{c}_j^{(d)}(x)$ be draw d from stage j's lognormal cost predictor. A full-panel cached prefix sets that stage's acquisition cost to $\epsilon = 1$ weighted-token unit. The acquisition is

$$A_{\mathrm{EEIPU}}(x) = \mathrm{EI}(x) \left[\frac{1}{128} \sum_{d=1}^{128} \frac{1}{\sum_{j=1}^{3} \widetilde{c}_j^{(d)}(x)} \right]^{\eta(C)}, \qquad \eta(C) = \max(0, 1 - C/B).$$

Independent stage models use independent normal draws, with common draws across candidates within a stage to reduce ranking noise. Objective EI is computed analytically from the quality GP. The one-unit acquisition floor does not add a charge to a physical cache hit.

Our persistent cache retains every observed prefix. Consequently, all materialized full-panel prefixes can discount the acquisition, while only Top5 prefixes bias candidate generation. This differs from restricting physical cache capacity to the best five workflows. The categorical Matérn-5/2 GPs, uniform legal completion search and normal Monte Carlo sampling are additional adaptations to the shared RAG domain.

G.5 LaMBO: modular regions and slowly moving allocation

LaMBO uses modular structure to reduce changes to earlier computations (Lin et al., 2021). The query and evidence modules are partitioned separately: their distinct categorical tuples are sorted and split at the midpoint, with the first group receiving the extra value when the count is odd. The two binary partitions define a structure-embedding tree with depths $(1, 1)$. Partitions remain fixed throughout search.

Five uniform complete workflows initialize a GP over the full one-hot configuration. For each tree region, the local candidate minimizes $1 - \mu_F(x) - 2\sigma_F(x)$ among its eligible unseen workflows. A lazy update keeps the previous workflow's modules before the first changed region fixed. Region selection conditions the current arm probabilities on the subtree determined by the preceding height. Independent random signs determine the next height, and the slowly moving bandit recursion updates multiplicative weights with learning rate 0.05. Only the selected workflow is executed; all local acquisition values are surrogate predictions rather than additional objective observations.

Finite exhaustion requires explicit handling. Regions without eligible unseen workflows are masked. If the conditioned subtree has no candidate, selection returns to the available root regions. If every lazy suffix is exhausted while unseen prefixes remain, a recorded root reset releases the prefix restriction. The adapter retains the fixed-partition recursion and omits the source paper's empirical discard/refine heuristic. Its tree depths are not refitted from test results or realized cost advantages.

LaMBO's preceding-configuration switching model and WTB's cumulative prefix availability describe different reuse conditions. This experiment reports the latter's physical calls and charges, including reuse of a prefix visited earlier than the immediately preceding workflow. A separate switching-cost diagnostic would require an explicitly specified module cost vector; it is not substituted for the measured optimization bill.

G.6 MSBO: native stage execution and cascade decisions

MSBO chooses between initiating a sample and continuing an existing sample (Torresi and Friederich, 2026). Here an evidence sample consists of native execution checkpoints after the first six actions, one for each training question. Let A_q be the grounded assessor's Boolean answerability judgment, E_q the materialized evidence-document set, and $G_q \subseteq E_q$ the distinct document identifiers cited in its grounded facts. A completed evidence stage supplies

$$z_1 = \frac{1}{150}\sum_q \mathbf{1}\{A_q = \text{true}\}, \qquad z_2 = \frac{1}{150}\sum_q \frac{|G_q|}{\max(1, |E_q|)}.$$

These are execution-derived proxies. They contain no gold answer, answer correctness or partial-panel F1. An invalid native assessment contributes its recorded false judgment and empty evidence claims. A missing required assessment is an execution failure rather than an invented proxy value.

Advancing a sample restores all six-action checkpoints and executes evidence retention, context ordering and answer generation. All 150 predictions must exist before the official answer F1 is calculated. This terminal observation alone can enter the training shortlist. A saved evidence sample can support multiple distinct suffixes under the common persistent execution service.

Five complete warmup workflows, each with a distinct evidence prefix, initialize the cascade. A warmup prefix is followed by its chosen terminal suffix. Subsequently, two independent RBF GPs predict z_1, z_2 from prefix features; the terminal GP predicts F1 from the full configuration and both proxies. Including configuration features in the terminal GP retains pipeline identity beyond the two aggregated evidence observations. For a new prefix, 256 normal Monte Carlo draws propagate both proxy uncertainty and terminal uncertainty. Advancing an observed sample conditions on its measured proxies. The candidate groups comprise at most 256 new-prefix paths and every unseen legal suffix of each saved sample.

The selection rule first maximizes Monte Carlo EI relative to the best completed training F1. If the largest EI is at most 0.005, it uses the source qUCB fallback. For terminal draws $Y^{(d)}(x)$ and their sample mean $\bar{Y}(x)$, the implemented single-candidate score is

$$U(x) = \frac{1}{256}\sum_{d=1}^{256}\left[\bar{Y}(x) + \sqrt{\frac{\pi\beta_{\text{MSBO}}(C)}{2}}\,|Y^{(d)}(x) - \bar{Y}(x)|\right],$$

$$\beta_{\text{MSBO}}(C) = 1 + 100\max(0, 1 - C/B), \qquad B = 32\text{M}.$$

Actual cumulative search cost is updated after a complete stage panel. There is no cost division in this acquisition, consistent with the authors' non-cost-weighted main configuration. Here resource awareness enters the stopping account and the stated exploration schedule.

The source implementation also occasionally selects a complete cascade. We retain its period of 1,000 acquisition decisions and map its 100 + initial-data cost epochs to 100 + 5 = 105 normalized units. The final full-path window is

therefore $B/105$. When $B - C \le B/105$, or the acquisition count is a multiple of 1,000, candidates are scored through the entire cascade posterior, even if a chosen prefix is already cached. The selected complete tuple is then fixed. A new prefix executes both stages; a materialized prefix proceeds directly to its selected suffix. If the evidence stage crosses the budget, its fixed suffix still finishes, with no intervening acquisition. This rule can cross both the resource budget and the ordinary stage-count ceiling, because it completes one already selected full-path decision.

The default ceiling is 256 completed workflows or 512 selected stages, subject to that full-path completion rule. Every physical stage is billed in order, including evidence samples never advanced. Primary results use the terminal boundaries defined in Appendix G.1. Additional stage snapshots retain paid evidence, completed-stage counts and unfinished sample identities. Intermediate or crossing observations cannot enter an earlier primary snapshot. Saved decisions and native checkpoints allow a partially executed stage or fixed suffix to resume without selecting against later data.

This adapter replaces continuous laboratory controls and measurements by finite RAG actions and two evidence proxies. It uses scalar-lengthscale RBF GPs and normal Monte Carlo draws in place of per-coordinate lengthscales and Sobol sampling. The five-workflow warmup replaces a dimension-scaled initial design. These adaptations preserve the start/advance decision and cascade uncertainty calculation while defining a new application of MSBO.

G.7 Surrogate computation and uniform-terminal control

The EEIPU and LaMBO GPs use Matérn-5/2 kernels on one-hot action vectors; MSBO uses RBF kernels. Each target is centered and scaled, with a minimum scale of 10^{-3}. A scalar lengthscale and observation-noise parameter are fitted by marginal likelihood when at least three nonconstant observations are available, using at most 40 optimization iterations. Log-lengthscale and log-noise bounds are $[-2.3, 3]$ and $[-13.8, -0.7]$, respectively; the diagonal numerical jitter is 10^{-9}. These are finite categorical surrogate choices. Surrogate fitting and acquisition are timed separately from task execution and are not converted into unobserved model-token charges.

Uniform-terminal search provides a complete-workflow allocation control. Exact descendant counts assign each legal terminal an integer rank. A seeded sample of distinct ranks is uniform over terminals, even when different first actions have different numbers of completions. The selected paths undergo the same paid 150-question training evaluation, three budget snapshots and validation selection. This differs from random-tree allocation, whose local child choices induce a different terminal distribution, and from the fifty frozen random workflows evaluated directly on test in Section M.8.2.

H Operational Execution and Backup Identities

The formal model represents a completed evaluation by an expanded path v_t, a terminal completion x_t extending it, and its reward. The legal-tree recurrence starts with the root. An event either evaluates an already expanded terminal or inserts one new child of an existing node. Inserting a child preserves prefix closure. The allocation predicate additionally requires unvisited children first and index maximality whenever all legal children have positive counts. Legality alone is not a definition of UCT.

H.1 Counts, Returns, and Birth Events

Inducting over the event list in (M.4) gives

$$N_t(p) = \sum_{s \le t} \mathbf{1}\{p \preceq v_s\}, \tag{23}$$

$$S_t(p) = \sum_{s \le t} \mathbf{1}\{p \preceq v_s\} f(x_s). \tag{24}$$

Every backed-up terminal extends p, so $0 \le Q_t(p) \le V(p) \le 1$ for positive counts. Subtracting the return sum from $N_t(p)V(p)$ proves (M.19) and $Q_t(p) = V(p) - D_t(p)/N_t(p)$. At a terminal, prefix-antichain legality forces all its completed returns to equal its fixed value. Hence its deficit is zero.

For internal p, partition backed-up events by $v_s = p$ or by the first action a after p in v_s. The former can occur only once, when a non-root internal node is inserted. The latter are exactly the child visits. This proves (M.20), with a birth deficit between zero and one. The local action stream at p contains only the latter events. If it has length n, its child counts are exactly the global backed-up child counts, while the parent count is n at the root and $n + 1$ at an already born non-root internal node. Projecting a global history onto this stream therefore preserves the index with a fixed offset $o \in \{0, 1\}$. This step accounts for progressive expansion rather than treating every internal node as a stationary bandit.

H.2 Query-Specific Execution and Conservation

For each query and terminal, saved depths form an initial segment under prefix completeness. Its largest saved depth exists because the path is finite. A demanded key is absent precisely when its depth exceeds that maximum. Summing its nonnegative weight proves equivalence between the set-difference bill and deepest-checkpoint suffix execution. Publishing a complete path preserves prefix completeness.

For one successful evaluation, disjointness of the old ledger and the newly demanded keys gives

$$\sum_{r\in M_{t+1}} w_r - \sum_{r\in M_t} w_r = \sum_{r\in \mathcal{D}(x_t)\setminus M_t} w_r.$$

The current and historical graphs remain equal because both receive the same union update from equal initial states. Telescoping proves (16). Nonreusable overhead adds $\sum_t h_t(x_t)$ and never enters this finite resource budget. The proof applies to the same events used for reward backup, independently of how they were selected. Distinct query identifiers make their resource sets disjoint.

For (5), apply the union accounting identity first to K, A and then to $K \cup A, B$, and cancel the intermediate cache weight. This allows A and B to overlap. The operational transition publishes $\mathcal{D}(x_t)$ regardless of the depth of v_t; a prefix of x_t outside the expanded ancestor chain enters the cache and leaves its tree statistics unchanged. The formal statement is `rollout_publication_without_backup`.

I Explicit Proof of the Tree Regret Bound

This section gives the constants used by the Lean main theorem. Let K_p be the number of legal children of p, $K = \max_p K_p$, and $D = \max_{x\in\mathcal{X}} |x|$. Choose $\delta > 0$ no larger than every positive gap Δ_{pa}. Finiteness supplies such a number; when every gap is zero, any positive δ suffices. This choice is made before quantifying over histories or horizons.

The penalty in (M.6) is bounded by

$$H = \gamma \sum_{r\in\mathcal{D}(X)} M_r.$$

For a target child count $m \in \mathbb{N}$, define

$$L_p(m) = \left\lceil \max\left\{ K_p + 1, \exp\left(\frac{(H+2)^2 m + 1}{c^2}\right), (8K_p \max\{1, c^2\} + 2)^2 \right\} \right\rceil, \tag{25}$$

$$F_p(m) = 2L_p(m).$$

Lemma 1 (Finite local exploration window)**.** *At an internal node, suppose means lie in $[0,1]$, penalties lie in $[0,H]$, unvisited children are taken first, and subsequent choices maximize the actual index with $c > 0$ and parent offset $o \in \{0,1\}$. After $F_p(m)$ local child choices, every child has more than m visits. Means and penalties may otherwise vary arbitrarily with the history.*

Proof. Initialization visits each child within the first K_p choices. Let $L = L_p(m)$ and suppose some child has at most m visits at local time $2L$. Throughout the interval $[L, 2L)$, its count is positive and at most m. The exponential term in (25) gives

$$c^2 \log(L + o + 1) > (H+2)^2 m.$$

Its exploration bonus therefore exceeds $H + 2$. Its index exceeds two, since its mean is nonnegative and its penalty is at most H. A selected child's mean is at most one and its penalty is nonnegative, so its bonus must exceed one. Thus each selected child has prior count at most $c^2 \log(2L + o + 1)$. Each of the K_p children can be selected at such counts at most $c^2 \log(2L + o + 1) + 1$ times.

The square term in (25) ensures

$$K_p\left(c^2 \log(2L + o + 1) + 1\right) < L.$$

For completeness, $\log z \le 2\sqrt{z}$ for $z > 0$, $2L + o + 1 \le 4L$, and $\sqrt{L} \ge 8K_p \max\{1, c^2\} + 2$ imply this inequality. There are L choices in the interval, contradicting the count bound. All arguments concern a finite observed history; no infinite continuation is assumed. □

I.1 Coverage and Disappearance of Cold Penalties

Define

$$C_0 = 1, \qquad C_{d+1} = 1 + \max_{p \in \mathcal{T}} F_p(C_d), \qquad T_{\text{cache}} = C_D. \tag{26}$$

We prove by remaining depth that C_d visits to a node suffice for every terminal below it to be visited. At depth zero, a positive visit evaluates that terminal. At an internal node, discarding its possible single birth event leaves at least $F_p(C_d)$ local choices. Lemma 1 gives each child more than C_d visits, and the induction hypothesis applies. The root count is the number of completed evaluations, so every terminal has been visited once $T \geq T_{\text{cache}}$.

Each such visit materializes all resource pairs for its terminal and complete panel. Therefore K_T covers $\mathcal{D}(\mathcal{X})$ after T_{cache}, and every cold forecast becomes zero. Before that time, normalization and nonnegative forecast weights bound it by H. This derives both the bound and disappearance from the execution model; neither is postulated as a property of an unknown error process.

I.2 Closing the Deficit Induction

Set $A_0 = B_0 = 0$. Given A_d, B_d, define

$$\begin{aligned} m_d &= \left\lceil \max\left\{1, (32A_d/\delta)^2, 8B_d/\delta\right\} \right\rceil, \\ \tau_d &= \max_{p \in \mathcal{T}} \max\{T_{\text{cache}}, F_p(m_d)\}, \\ A_{d+1} &= K\left(A_d + \frac{4c^2}{\delta^2}\right), \\ B_{d+1} &= 1 + K(B_d + \tau_d + 1). \end{aligned} \tag{27}$$

We prove simultaneously for every finite legal event history and every node of remaining depth at most d that

$$D_t(p) \leq A_d \log(N_t(p) + 1) + B_d. \tag{28}$$

The depth-zero statement follows from the exact terminal mean. Assume the result for all child subtrees and all history prefixes. For $n \geq m_d$, the inequalities $\log(n+1) \leq 4\sqrt{n}$ and (27) give

$$\frac{A_d \log(n+1) + B_d}{n} \leq \delta/4.$$

After τ_d local choices at an internal node, every child exceeds this count threshold by Lemma 1. Global time is at least local time, so the cold penalties have also disappeared. An optimal child a^* has

$$Q_t(pa^*) \geq V(p) - \delta/4, \qquad Q_t(pa) \leq V(pa)$$

for any child a. If a child with positive gap is selected, index maximality and the nonnegative optimal-child bonus imply that its own bonus is at least $\delta/2$. Consequently its prior count is at most

$$\frac{4c^2 \log(N_t(p) + 1)}{\delta^2}.$$

Monotonicity of counts and of the logarithm bounds all late selections by the final parent count. Counting the at most τ_d early selections and the last late increment yields

$$N_T(pa) \leq \tau_d + \frac{4c^2 \log(N_T(p) + 1)}{\delta^2} + 1 \tag{29}$$

for each positive-gap child. Zero-gap children contribute nothing to $N_T(pa)\Delta_{pa}$.

Insert (29) and the child induction hypotheses into (M.20). Use $0 \leq \Delta_{pa} \leq 1$, $N_T(pa) \leq N_T(p)$, the birth bound one, and at most K children. This gives exactly (28) with the constants in (27). At the root, $N_T = T$ and $D_T = R_Q(T)$, proving Theorem M.2. All constants were fixed before the arbitrary history and are independent of T.

Ordinary UCT uses the same induction with $H = 0$ and $T_{\text{cache}} = 0$. Proposition 2 transfers that bound to the legacy direct-edge estimator without adding a cache-coverage delay: initialized direct edges are already published. The current first-unpaid estimator instead uses its domination by the complete cold bill and the coverage argument. For the full-cold policy the explicit coverage constants can be enormous, particularly at greater depth. The proof establishes a finite instance-dependent bound, not efficient worst-case discovery.

I.3 Reading the Lean Proof in Dependency Order

The following steps identify the actual proof obligations. The English guide `theory/lean/AgentUCTProofGuideEN.lean` follows the same order; its complete-proof companion `theory/lean/AgentUCTCompleteProofsEN.lean` contains all definitions and proof bodies in dependency order, with English step comments and source-module locations. Each `#check` prints the full declaration and its explicit hypotheses; it does not add a theorem.

1. The backup recurrence gives counts and return sums. `operational_mean_deficit` converts the actual mean to $V - D/N$ for positive N. The parent partition retains the single birth deficit and the local-stream offset; these are operational identities.

2. Nonnegative normalized completion laws and a fixed price envelope give a history-independent penalty bound. It constrains predictions, not physical receipts or admission of model calls.

3. `finite_window_explores` proves the finite-window contradiction. `finite_explicit_window_explores` supplies the explicit exponential and square thresholds. `actual_children_explored` transfers the statement to the actual local action stream.

4. `actual_subtree_coverage` inducts on remaining depth, retaining the possible birth visit. `actual_restorable_coverage` converts complete terminal publications to persistent executable resources.

5. `coldForecastPenalty_after_coverage` derives zero penalties from empty missing-resource sets. Bounded penalties alone establish exploration, and do not discharge this obligation.

6. `logarithmic_deficit_small` divides the child induction bound by its positive count to obtain $D/N \le \delta/4$. The optimal child then has mean at least $V(p) - \delta/4$.

7. `actual_suboptimal_child_count` combines this mean bound, zero penalties, index maximality and the bonus-square identity. It retains the early threshold and the last increment.

8. `parent_deficit_log_bound` sums the parent decomposition; `actual_subtree_log_deficit` closes the remaining-depth induction simultaneously for every history prefix. Root count and deficit identities give the cumulative training-quality bound.

9. `onlineMean_envelope` bounds the online price mean. `sampledLaw_mass` retains multiplicity and requires a positive sample count. They discharge estimator premises of `online_mean_shared_root_log_bound`; fixed scores and successful persistent publication remain execution assumptions.

Table 29: Proof-to-claim map. The local recurring-cost counterexample and conditional confidence results retain their own hypotheses; they are not the proof of the operational tree rate.

Paper result	Representative Lean declaration in namespace `AgentUCTPaper`
Full-panel compression	`full_panel_residual_quotient`, `full_panel_history_rectangular`
Demand-profile compression	`demand_group_cost_quotient`, `demand_group_publication`, `demand_separating_merge_loses_information`
Query-selective information loss	`GraphWitness.no_count_only_optimal_policy`, `GraphWitness.budget_feasibility_reverses`
Actual reward backup	`replayBackup_visits`, `replayBackup_mean`, `operational_mean_deficit`
Graph accounting and suffix	`replayMetered_cold_conservation`, `coldWork_eq_deepest_suffix`
Tree/stream correspondence	`localBefore_parent_count`, `projected_index_max`
Exploration and coverage	`actual_children_explored`, `actual_restorable_coverage`
Online-price tree rate	`checkpoint_conditioned_shared_root_log_bound`, `online_mean_shared_root_log_bound`
Fixed-price special case	`cold_forecast_uniform_root_log_bound`
Shared fixed-panel family	`shared_cache_uniform_root_log_bound`, `fixed_panel_family_log_bound`
Completion index and budget	`completion_index_error`, `own_cache_budget_coverage`
Same-state pricing benefit	`priced_choice_tradeoff`, `physical_saving_certificate`, `actual_net_gain_certificate`
Exclusive trajectory costs	`own_cache_cost_difference`
Own-cache cumulative objective	`own_cache_policy_regret_bound`, `own_cache_cost_difference_common_overhead`
Recommendation and endpoint	`validation_shortlist_gap`, `equal_quality_positive_endpoint_gap`
Full cost and its errors	`referenceColdCost_resource_form`, `full_reference_cost_error`
Prediction and optimism	`full_cost_mean_unique_mse_minimizer`, `selected_net_gap_from_cost_radii`
Legacy direct-edge reduction	`next_edge_allocation_iff_uct`, `next_edge_uniform_root_log_bound`, `legacyContinuationCold_common_shift`
First unpaid execution	`nextUnpaidBill_le_complete`, `dominated_cold_shared_root_log_bound`
Coupled trajectories	`resolvedTreeHistory_valid`, `next_edge_coupled_history_eq_uct`, `next_edge_coupled_state_eq_uct`
Local recurring-cost failure	`next_edge_linear_regret`

J Other Objectives and Estimator Proofs

J.1 Independent Cache Trajectories and Endpoints

Each comparator reward is at most V^*, so its cumulative reward advantage is at most $R_Q(T)$. The actual cold bill is at most W and the comparator cold bill is nonnegative, even though they use different caches. This proves (18). Equal recurring-overhead totals cancel before applying the cold-budget bound.

The best observed training reward is at least the average observed reward. Subtracting from V^* gives the simple-gap bound. After terminal coverage, an observed training maximizer is a global table maximizer. For (19), let x_g maximize validation and let x_f be the training-ranked head. The head belongs to every nonempty top-k shortlist, so $g(x_{\mathrm{end}}) \geq g(x_f)$. Therefore

$$\begin{aligned} g(x_g) - g(x_{\mathrm{end}}) &\leq f(x_g) + \epsilon - f(x_f) + \epsilon \\ &\leq R_Q(T)/T + 2\epsilon. \end{aligned}$$

The uniform table difference is a hypothesis. For a counterexample without it, two fully observed workflows with training scores $(1, 0)$ and validation scores $(0, 1)$ give validation regret one after training top-1 selection.

For endpoint utility, direct subtraction gives

$$J^\pi - J = \left(q_{\mathrm{rep}}(x^\pi_{\mathrm{end}}) - q_{\mathrm{rep}}(x_{\mathrm{end}})\right) + \frac{\lambda_J}{C_{\mathrm{ref}}}(C_{\mathrm{opt}} - C^\pi_{\mathrm{opt}}).$$

Equal quality and a strictly larger actual bill give a strictly positive gap when $\lambda_J, C_{\mathrm{ref}} > 0$. Cumulative regret and endpoint utility thus require distinct interpretations.

J.2 Cost Targets and Error Bounds

For the resource formula (M.12), exchange the two finite sums in (M.6). The coefficient of each forecast weight is the reference probability that this resource is demanded and absent from the current graph. Shared-resource dependence never enters this operation.

To isolate the effect of publication, partition $\mathcal{D}(x) \setminus K$ into the disjoint sets $\mathcal{D}(x) \setminus (K \cup A)$ and $(\mathcal{D}(x) \cap A) \setminus K$. Subtracting the first bill leaves exactly the second. Averaging with the same law and weights proves (6). This argument formalizes sequential cache accumulation and its effect on prediction without treating a prediction as physical execution; see `coldWork_sequential_union`, `referenceColdCost_publication_savings`, and `referenceColdCost_cache_antitone`.

For (1), expand the square around the full conditional mean m_t. The cross term is zero because $\mathbb{E}[b_t(X) - m_t] = 0$. The remaining excess is $(z - m_t)^2$, which vanishes exactly at $z = m_t$. The formal finite-sum theorem needs normalization; nonnegativity makes the normalized weights a probability law.

To prove Proposition 1, insert $\sum_x \rho_C(x) b_t(x)$ between the forecast mean and the actual mean. The first difference is bounded by the reference-weighted bill errors. Apply the triangle inequality within each cold resource sum and to the overhead. Bound the second difference by $B_{\max} \sum_x |\rho_C(x) - \rho_{\text{exec}}(x)|$. The estimate explicitly includes both sources of error.

Let $|\widehat{b} - b| \le e$ and $b \ge 0$. Then $0 \le \underline{b} = \max\{0, \widehat{b} - e\} \le b$ and $b - \underline{b} \le 2e$. These facts prove (9). If a maximizes that optimistic index and a^* maximizes true net value, then

$$\begin{aligned} \mu_{a^*} - \gamma b_{a^*} &\le Q_{a^*} + r_{a^*} - \gamma \underline{b}_{a^*} \\ &\le Q_a + r_a - \gamma \underline{b}_a \\ &\le \mu_a + 2r_a - \gamma b_a + 2\gamma e_a. \end{aligned}$$

The last step uses the selected arm's reward upper-error bound. This proves the stated conditional net-gap inequality without asserting that tree means satisfy those reward confidence conditions.

J.3 Reduction and Recurring-Cost Counterexample

A positive child count witnesses an earlier complete event whose expanded path passes through that child. Its terminal completion extends the child and has materialized the child's entire panel demand. Persistent publication retains that demand, so its current cold bill is zero for every nonnegative predictor. The same holds for its parent. Equation (7) is zero, and the actual initialized-child index comparisons become exactly ordinary UCT comparisons. Unvisited-first clauses are identical as well. This proves equivalence of the whole allocation predicates on a given history.

For trajectory equality, let $\mathcal{E}_{\text{next}}(h)$ and $\mathcal{E}_{\text{UCT}}(h)$ be the complete events that can extend history h while satisfying the same fixed reward table, legal expansion protocol, and respective allocation rule. The predicate equivalence gives $\mathcal{E}_{\text{next}}(h) = \mathcal{E}_{\text{UCT}}(h)$. Use the same initial tree and cache and a shared resolver $\Phi(h, \mathcal{E})$ that returns an admissible event or stops. The resolver fixes expansion, rollout completion, tie-breaking, evaluation outcomes including receipts, and stopping; a common random tape can be fixed as part of it. Equal histories give equal event sets and the same output of Φ. Induction therefore gives equal event histories at every step, hence equal reward backups, expanded trees, and cache graphs. The Lean development proves both resolver validity and this coupled equality. Rollout coupling is necessary: expanding the same node can lead to different leaves and rewards when its completion choices differ.

For historical continuation costs, each terminal extending p demands a superset of p's resources. Subtracting the same cached set preserves inclusion, and nonnegative weights give $\widehat{d}_t(x) \ge \widehat{d}_t(p)$. Remove the inactive clipping and use $\sum_x \rho_C(x) = 1$ to pull the parent term outside the expectation. This proves the common shift. With a common omitted tail the same algebra applies; action-dependent tails introduce the exact pairwise difference given in the main text.

For (8), apply the ordinary-UCT quality bound to the next-edge allocation predicate. The comparator's reward at every step is at most V^*. Each trajectory is charged against its own cache, and cancellation of equal recurring-overhead totals leaves a cost difference at most W. Multiplying by $\eta \ge 0$ proves the bound. The complete composed statement is `next_edge_own_cache_log_bound`; it quantifies the gap and constants before all actual and comparator histories. For the local approximation relation, full net gap equals next-edge net gap plus η times chosen-tail minus comparator-tail at each step. Sum these identities and use the assumed tail-variation bounds, as in `next_edge_tail_regret_transfer`. A logarithmic local proxy bound remains a separate premise.

For the recurring-cost example, initialize arm A then arm B. Their immediate-cost net values are 0.4 and 0.6, while their full-cost net values are 0.4 and 0.3. The next-edge UCB rule therefore treats A as the inferior proxy arm with gap 0.2. At any subsequent selection of A, exact deterministic reward means and index maximality imply $0.2 \le \sqrt{2\log(t+1)/N_A(t)}$. Counting such selections gives a logarithmic upper bound on $N_A(T)$. In particular the conservative bound $N_A(T) \le 400 \log T + 4$ holds for $T \ge 1$; for $T \ge 2$ the direct count bound is already stronger.

Since full-cost regret is $0.1(T - N_A(T))$, the claimed lower bound follows. The preserved Lean proof obtains the same conservative constants via its general local-regret counting lemma.

J.4 Cache Compression and Information Loss

For Proposition M.1, exchange the finite sums over qids and demanded prefixes. This gives the residual bill in (M.8). Full-panel publication adds every qid incidence for each demanded prefix, so exactly those residual bills become zero. The product identity $(Q \times S) \cup (Q \times P) = Q \times (S \cup P)$ proves rectangularity by induction over the publication history. Lean checks these statements in `full_panel_residual_quotient`, `full_panel_residual_update`, and `full_panel_history_rectangular`.

For Proposition 3, enumerate the two resource sets in the stated witness. Both hit-count vectors and both full-panel residual-bill vectors coincide. The partial-panel bills are $(0, 1)$ and $(1, 0)$. A deterministic function of the identical summary chooses the same action in both states, and therefore loses one unit in one of them. If it chooses u with probability p, expected excess costs are $1 - p$ and p, whose maximum is at least $1/2$. A common estimate z of u's bill has errors $|z|$ and $|z - 1|$, again with maximum at least $1/2$. Finally, the unit recurring bill makes total costs $(1, 2)$ and $(2, 1)$, proving the budget claim. The 12 checked declarations and assumption review are recorded in `theory/evidence/graph-information/REVIEW.txt`. These are information and one-decision results; no multi-round policy-regret lower bound is inferred.

J.5 Completion Decisions and Shared Panels

For Theorem M.1, each allowed panel is a union of profile classes. Exchange the finite sums over queries, groups, and prefixes to obtain Equation (M.7). Publication includes either every member of a class at a prefix or none, so the displayed zeroing update is closed on the summary. For necessity within partition-based residual sums, let a merged class contain $q \in Q_j$ and $r \notin Q_j$. With unit prices, compare a residual vector missing only q to one missing only r. Every group total is identical; the panel bills are one and zero. No decoder of those totals recovers both. One-step prefixes make both caches prefix-complete. The Lean module `TreeDemandQuotient` proves the finite-sum identity, publication rule, profile criterion, and decoder impossibility separately. The necessity statement does not rule out stronger encodings or compressions specific to a restricted set of reachable histories.

For Proposition M.2, unpriced maximality gives $U_b - U_a \geq 0$ and priced maximality gives $U_b - U_a \leq \gamma(\widehat{b}_b - \widehat{b}_a)$. Positive γ implies $\widehat{b}_a \leq \widehat{b}_b$. The triangle inequality gives $b_b - b_a \geq \widehat{b}_b - \widehat{b}_a - \epsilon_a - \epsilon_b$. Applying the reward-error bounds as well proves (12). For (14), apply sequential cache accounting to $A \cup B$ in both orders and subtract the two expressions. All these arguments are checked in `TreeDecisionValue`. Its numerical example checks only the two displayed one-decision cases, with no hidden claim of a legal full-tree counterexample or of calibrated UCT confidence bounds.

Equation (M.14) is a rearrangement of the two indices. For Proposition 5, insert $\widehat{b}_a$ and $\widehat{b}_{a^*}$. Estimated-index maximality removes the middle difference; the remaining terms are bounded by $\gamma\epsilon_a$ and $\gamma\epsilon_{a^*}$. Lean also checks the finite-sum version in `completion_index_cumulative_error`. The next-edge reversal follows by subtracting the common edge bill. The scaled information loss follows from $\max\{H(1-p), Hp\} \geq H/2$. Proposition 6 sums each policy's own costs, with nonnegative prefix costs ensuring every earlier prefix also fits. These declarations are in `TreeFiniteBudget`.

For Theorem 1, write a local cache as $K_t^{\text{own}} \cup E_t$, where E_t contains publications made outside the local tree. The cold penalty is nonnegative and at most H_j because the completion law is normalized and all resource predictions are nonnegative. Construct the same bounded tree run as in the single-panel proof. At its finite coverage threshold, local publications alone contain every required resource. Their inclusion in $K_t^{\text{own}} \cup E_t$ zeros the cold penalty. The original subtree induction then applies without alteration. The external set may depend on the local history; no independence assumption is used. Summing the resulting local bounds and using $T_j \leq T$ gives the family bound, with nonnegative A_j. `TreeSharedCache` checks both statements.

The first unpaid estimator uses an ordered resource list for each query. `nextUnpaidBill` skips cached resources and nonpositive predicted prices; otherwise it returns the current price. Induction gives a nonnegative value bounded by `orderedColdBill` for nonnegative predictions. Summing over queries and averaging under the same nonnegative completion probabilities preserves this inequality. Full coverage zeros the estimate. The theorem `dominated_cold_shared_root_log_bound` proves the whole-tree bound for any nonnegative penalty bounded above by the complete cold forecast. Its proof replaces the full-forecast bound by transitivity and uses that domination to zero the penalty at coverage. The immediate-child reduction is not used. `nextUnpaidBill_cached_edge_counterexample` checks the saved-edge example in the cost-estimator analysis.

J.6 Reachable Separation and Generated Workflows

For Proposition 4, both demand lists contain four unit-price resources and start from the empty set. Direct set subtraction gives one residual unit at each prefix over the three-query universe. Restricting the next demand to A gives $(0,1)$ and $(1,0)$. Adding the common unit cost proves the budget statement. `ReachableDemandSeparation` checks reachability, warm-up bills, residual equality, ranking and affordability.

For Proposition 7, induct on the realized demand list. The induction step uses $c_w(K,A) + c_w(K \cup A, B) = c_w(K, A \cup B)$. Adding the recurring-work sum yields the complete accounting identity. Expanding both sides of Equation (20) and normalizing the conditional execution law proves the two-stage expression, with arbitrary dependence on the generated child.

For Proposition 8, optimistic bounds give $v_t(a_t^*) - \gamma m_t(a_t^*) \le U_t(a_t^*) - \gamma L_t(a_t^*)$. Maximality bounds this by the chosen index. Subtracting the chosen true net value leaves at most $s_t + \gamma e_t$; finite summation proves the result. `DynamicCompletion` checks the identities, this bound, zero-weight padding, and the fresh-unit-resource sequence whose total bill is T. Its statements quantify over the realized cache state and do not require a predeclared finite universe of generated programs.

For checkpoint-conditioned predictions, finite summation preserves $0 \le \widehat{w}_{t,r} \le \bar{w}_r$. Averaging under any nonnegative continuation law preserves the same domination. Apply the shared-cache theorem with the fixed envelope $\bar{w}$: its coverage threshold and constants still apply, and coverage zeros every dominated live penalty. The declaration `checkpoint_conditioned_shared_root_log_bound` checks this complete composition, including history-dependent prices and continuation laws.

K Formal Verification and Reproduction Map

The project pins Lean 4.24.0 and mathlib v4.24.0. Its unified entry is `theory/lean/AgentUCTPaper.lean`. The original operational development and archived conditional stochastic results remain compiled. `TreeGraphInformation`, `TreeFiniteBudget`, `TreeSharedCache`, and `TreeNextUnpaid` cover the compression witness, budget conditions, shared-panel rate, and first unpaid cost. The current revision also includes `TreeDemandQuotient`, `TreeDecisionValue`, `DynamicCompletion`, `ReachableDemandSeparation`, and `OnlineMean`. The integrated build, four language-server checks, and dependency audit pass; evidence is under `theory/evidence/reviewer-revision` and `theory/evidence/p0-p1`. The latter contains 17 added theorem checks and three completed language-server requests with zero diagnostics. The audited dependencies are Lean's standard `propext`, `Classical.choice`, and `Quot.sound`; there are no admitted proofs or project-defined axioms.

The main theorem quantifies its instance gap and constants before all finite event histories. It discharges child-deficit bounds by induction, without assuming a concentration event, logarithmic drift budget, or desired root-regret conclusion.

The online implementation mapping uses `OnlineMean`, whose envelope and sampled-law lemmas connect incremental means and finite completion samples to `online_mean_shared_root_log_bound`. The integrated build is recorded in `theory/evidence/algorithm/lean-build.log`; dependencies are listed in `theory/evidence/algorithm/lean-axioms.log`. The source declarations define the formal claims. Python refinement is not machine-verified. The historical event audits in `evidence/acquisition_update` and `evidence/theory_alignment` concern their recorded runtime revisions. Current runtime regressions and receipt evidence are under `evidence/main-pair-v1`; full reconstruction of the current experimental trajectories remains a separate post-run analysis. Paths are relative to the workspace root.

The shared-cache theorem permits a fixed finite family of query panels with separate reward statistics. Its state-conditioned extension requires successful complete evaluations, persistent local cache entries, a fixed finite nonnegative prediction envelope, and deterministic reward tables. Arbitrarily changing the queries behind a shared reward mean, eviction, stochastic rewards, and action-dependent recurring penalties lie outside that theorem. The archived conditional stochastic results retain their stated assumptions.

Current proof inventory. This revision builds with 3305 Lake jobs, checks 96 declaration references in `AgentUCTProofGuideEN`, and audits all 385 public theorems in the 45 mathematical source files. Dependencies are `propext`, `Classical.choice` and `Quot.sound`; the source scan finds no admitted proofs or custom axioms. Logs and inventory are under `theory/evidence/proof-guide-en-20261004`. The English complete-proof file assembles the 45 modules; the original sources remain the editing locations.

K.1 Proven Results and Extension Directions

The proofs establish finite-tree quality rates under fixed rewards and persistent successful publication, exact execution accounting and full-panel cost quotients, and same-state pricing certificates with explicit error assumptions. Training, conditional validation and finite-history dynamic results retain their stated assumptions.

Extensions to stochastic rewards, publication failure or eviction require new operational models. Certified forecast errors and same-state costs of unselected candidates would connect the certificates to measured savings. Test generalization, growing-tree quality rates and Python refinement require separate distribution, comparator and transition models.

Equal-budget test F1 remains an empirical comparison. The rate counts completed evaluations, including free replays. A frozen empty-cache penalty need not vanish after coverage; boundedness alone does not transfer the theorem to that ablation. Extensions are assessed separately.

L AI-Assisted Development

OpenAI Codex assisted with theory formulation, proof drafting and Lean implementation, experimental-protocol discussion, runtime code, saved-result analysis, simulated reviewer feedback, and manuscript revision. This use included methodological and experimental-design work. Retained development logs record model identifiers `gpt-6-astra` and `gpt-6.1-sol`, and Codex client versions 0.159.0, 0.159.2 and 0.160.0. These are recorded identifiers, not inferred model release numbers.

Selected original design prompts and English translations are supplied in `evidence/main-pair-v1/ai-assistance-disclosure-20261004.json`. They requested analysis of graph compression and the rationale for cost-aware allocation; construction of the full-evaluation cost estimator; treatment of cache accumulation during rollout; actual-cost snapshot reporting; uniform batch execution; and validation-selected test comparisons across budgets. The inventory covers 15 retained development sessions and identifies the observed model set at session level. It is not a complete transcript or an inventory of unrecorded interactions.

The subsequent manuscript restructuring used the user-requested `nature-writing` 1.5.0 and `nature-figure` 2.8.0 guidance from https://github.com/Yuan1z0825/nature-skills. The method schematic was programmatically drawn with Matplotlib; it contains illustrative symbols, not generated experimental observations or image-model artwork. The disclosure file records the additional budget-analysis, token/cache reporting, paired statistical analysis, configuration-table and visual-reference requests. Supplementary prose also used `write-like-me` 0.1.1 for the author's requested direct style.

The delivered Lean sources and compiler logs support formal statements. Saved evaluations and receipts support empirical claims. The authors remain responsible for accuracy, citations and disclosure.

References

DeepSeek. 2026. DeepSeek-V4-Pro API Documentation and Change Log. Accessed 4 October 2026; model identity reference, no experimental result. https://api-docs.deepseek.com/updates/

Abdelmajid Essofi, Ridwan Salahuddeen, Munachiso Nwadike, Elnura Zhalieva, Kun Zhang, Eric Xing, Willie Neiswanger, and Qirong Ho. 2024. Reducing Hyperparameter Tuning Costs in ML, Vision and Language Model Training Pipelines via Memoization-Awareness. arXiv:2411.03731 https://arxiv.org/abs/2411.03731

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 1762–1777. doi:10.18653/v1/2023.acl-long.99

Yang Li, Hai Liu, Dian Shao, Yu Wang, Xiyu Chen, Sergey Volkov, Bozhi Wang, Ziyu Sun, Sihang Liu, Ye Luo, and Xiaowei Zhang. 2026. Agent-UCT: Upper Confidence Bounds Applied to Trees for Agentic Workflow Optimization with Cost-Awareness. arXiv:2607.24162 https://arxiv.org/abs/2607.24162v2

Chi-Heng Lin, Joseph D. Miano, and Eva L. Dyer. 2021. Bayesian Optimization for Modular Black-Box Systems with Switching Costs. In *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence* (Online) *(Proceedings of Machine Learning Research, Vol. 161)*, Cassio de Campos and Marloes H. Maathuis (Eds.). PMLR, 1024–1034. https://proceedings.mlr.press/v161/lin21c.html

OpenAI. 2026. GPT-5.5 Model Documentation. OpenRouter model identifier: openai/gpt-5.5; accessed 4 October 2026. https://developers.openai.com/api/docs/models/gpt-5.5

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 14918–14937. doi:10.18653/v1/2023.emnlp-main.923

Luca Torresi and Pascal Friederich. 2026. Multi-stage Bayesian optimisation for dynamic decision-making in self-driving labs. *Digital Discovery* 5 (2026), 1900–1912. doi:10.1039/D5DD00572H

Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query Expansion with Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 9414–9423. doi:10.18653/v1/2023.emnlp-main.585

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2024. AFlow: Automating Agentic Workflow Generation. arXiv:2410.10762 https://arxiv.org/abs/2410.10762

Yuyu Zhang, Mohammad Taha Bahadori, Hang Su, and Jimeng Sun. 2016. FLASH: Fast Bayesian Optimization for Data Analytic Pipelines. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 2065–2074. doi:10.1145/2939672.2939829